Laboratorul 1 - Introducere în programarea paralelă cu Pthreads

Responsabili: Radu Ciobanu, Florin Mihalache, Andrei Damian, Delia Stuparu, Dragoș Ușurelu

Programarea paralelă

În mod tradițional, programele pe care le-ați implementat până acum au fost scrise pentru **calcul serial**. Astfel, o problemă care trebuia rezolvată era împărțită într-un set discret de instrucțiuni, care erau executate pe un singur procesor în mod secvențial, una după cealaltă. La un moment dat de timp, o singură instrucțiune putea fi executată.

De cealaltă parte, **programarea paralelă** reprezintă utilizarea simultană a mai multor resurse de calcul pentru a rezolva o problemă. Din punct de vedere al pașilor de rezolvare, problema trebuie întâi împărțită în componente discrete care pot fi rezolvate concurent. Apoi, fiecare astfel de componentă trebuie mai departe împărțită în câte un set de instrucțiuni. În final, pentru a obține paralelism, instrucțiuni ale componentelor diferite ale problemei se pot executa simultan pe procesoare diferite. Pentru a se putea realiza acest lucru, este necesar un mecanism de coordonare a execuției diferitelor componente ale problemei.

Pentru a putea paraleliza în mod eficient o problemă, ea trebuie să poată fi împărțită logic în componente separate care pot fi executate simultan, iar durata execuției paralele a acestor componente trebuie să fie mai mică atunci când avem mai multe resurse de calcul decât atunci când avem o singură astfel de resursă. Pentru a putea executa un program paralel, este necesar să avem fie o mașină de calcul cu mai multe procesoare/core-uri, fie un număr arbitrar de astfel de mașini de calcul conectate printr-o rețea (dar aici extindem conceptul de programare paralelă către **programarea distribuită**).

Atunci când implementăm un program paralel, trebuie să ținem cont de mai multe considerente de design, cum ar fi:

- cum partiționăm problema?
- cum balansăm încărcarea?
- cum realizăm comunicația între componentele care rulează în paralel?
- ce dependențe de date avem?
- cum sincronizăm componentele paralele ale programului nostru?
- cât de mare este efortul de a paraleliza o problemă?

De-a lungul acestui semestru, vom încerca să adresăm cât mai multe din aceste probleme.

Fire de execuție

Un fir de execuție (sau **thread** în engleză) este definit ca un flux independent de instrucțiuni care pot fi planificate de către sistemul de operare. Din punct de vedere al unui programator, un fir de execuție poate fi descris cel mai bine ca o funcție care rulează independent de programul principal, iar un program paralel (cu mai multe fire de execuție, sau **multi-threaded**) poate fi privit ca toată mulțimea de astfel de funcții care pot fi planificate să ruleze simultan și/sau independent de către sistemul de operare.

Atenție! Există o distincție foarte importantă între conceptul de **proces** și cel de **thread**! Veți intra în detaliu la alte materii, dar, în acest moment, este foarte important de reținut faptul că un proces este o instanță de rulare a unui program (și deci **două procese distincte nu partajează spațiul de adrese**, care include stiva de program, variabile, date, etc.), pe când un thread este o unitate de lucru a unui proces (deci **mai multe thread-uri pot avea acces partajat la variabile și alte date**).

Datorită faptului ca firele de execuție ale aceluiași proces partajează resurse, modificările făcute de către un thread asupra acelor resurse (cum ar fi, de exemplu, închiderea unui fișier) vor fi observate de toate thread-urile acelui proces. Mai mult, doi pointeri cu aceeași valoare referă aceleași date, iar scrierea și citirea în/din aceeași zonă de memorie este posibilă, dar necesită sincronizare explicită din partea programatorului (mai multe detalii despre ce înseamnă sincronizarea și cum se realizează veți afla în laboratorul 2).

În general, programele care pot beneficia de implementare multi-threaded au câteva trăsături comune:

- conțin componente computaționale care se pot executa în paralel
- au date pe care se poate opera în paralel
- se blochează ocazional așteptând după I/O
- trebuie să răspundă la evenimente asincrone
- anumite componente de execuție au o prioritate mai mare decât altele.

Pthreads (thread-uri POSIX)

La începutul programării paralele, producătorii de hardware își implementau propriile versiuni de thread-uri, care difereau considerabil între ele, ceea ce ducea la dificultăți în dezvoltarea de aplicații multi-threaded portabile. Din acest motiv, o interfață standardizată de programare multi-thread a fost necesară, iar acest lucru a fost specificat, pentru sisteme UNIX, de către standardul IEEE POSIX 1003.1c în anul 1995. Astfel, implementările de thread-uri care aderă la acest standard sunt denumite thread-uri POSIX, sau Pthreads, iar astăzi majoritatea furnizorilor de hardware oferă și Pthreads.

Din punct de vedere al unui programator, Pthreads sunt definite ca o mulțime de tipuri și funcții pentru limbajul C, implementate într-un header numit **pthread.h** și o bibliotecă numită **pthread** (deși, în unele implementări de Pthreads, această bibliotecă poate fi inclusă într-o altă bibliotecă).

Implementarea unui program paralel folosind Pthreads

Compilare și rulare

Pentru a compila un program care folosește Pthreads, va trebui să link-ăm biblioteca de Pthreads. Rularea programului se realizează ca pentru orice alt program C. De exemplu, dacă folosim compilatorul GCC, putem compila și rula un program Pthreads astfel:

```
gcc -o program program.c -lpthread
./program
```

Atenție! Ca să implementăm un program care folosește Pthreads, trebuie să includem header-ul **pthread.h**.

Crearea și terminarea thread-urilor

Într-un program C cu Pthreads, inițial există un singur fir de execuție, numit thread-ul principal (**main thread**). Oricare alt fir de execuție trebuie creat și pornit în mod explicit de către programator, acest lucru făcându-se prin intermediul funcției **pthread_create**, care poate fi apelată de oricâte ori și de oriunde din cod. Funcția are următoarea semnătură:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);
```

Parametrul *thread* (de tip *pthread_t*) reprezintă un identificator pentru noul thread returnat de această funcție, ce poate fi apoi folosit pentru a opri sau a referi thread-ul. Parametrul *attr* este utilizat pentru a seta diferite atribute pentru firul de execuție care se creează, și poate fi pus pe NULL dacă se dorește păstrarea valorilor implicite. *start_routine* este un pointer la funcția care va fi executată pe firul de execuție nou-creat la pornirea sa. În final, prin intermediul parametrului *arg*, putem trimite un singur parametru către funcția de thread, care trebuie pasat prin referință ca un pointer de tipul *void* (dacă nu se dorește trimiterea unui parametru, putem lăsa NULL). Funcția returnează 0 dacă thread-ul nou s-a creat și s-a pornit cu succes, sau un cod de eroare în caz contrar.

Un thread creat astfel poate la rândul lui crea alte thread-uri, neexistând o ierarhie sau dependență între ele. Numărul maxim de thread-uri care poate fi creat de un proces depinde de implementarea Pthreads, dar în general nu se recomandă să avem un număr de fire de execuție mai mare decât numărul de core-uri de pe mașina pe care rulăm, din motive de overhead (aceasta este o discuție mai lungă pe care vă recomandăm să o purtați cu asistentul vostru).

Funcția pe care un thread o execută atunci când apelăm **pthread_create** trebuie să aibă următoarea semnătură:

```
void *f(void *arg) {
    [...]

    pthread_exit(NULL);
}
```

Parametrul *arg* al funcției de mai sus este primit de funcția **f** la execuția ei pe un thread nou atunci când se apelează **pthread_create**, fiind echivalentului parametrului *arg* de la **pthread_create**. Limitarea dată de faptul că putem trimite un singur parametru funcției de thread poate fi rezolvată prin crearea unei structuri cu oricâți membri, pe care o putem apoi da ca parametru prin referință. Funcția **pthread_exit** trebuie apelată pentru a termina thread-ul care o apelează, și primește ca parametru fie NULL, fie o valoare de retur care va fi pasată mai departe.

În momentul în care apelăm **pthread_create**, firul de execuție nou-creat va rula în paralel cu thread-ul principal. Acest lucru înseamnă că tot codul de după apelul funcției **pthread_create** se va executa în paralel cu codul noului thread. Dacă dorim ca un fir de execuție să aștepte terminarea unui alt thread, putem realiza acest lucru prin funcția **pthread_join**. Prin apelul acestei funcții, ne putem asigura că thread-ul apelant se blochează până când celălalt fir de execuție își termină procesarea (adică termină de executat funcția de thread). Funcția **pthread_join** are următoarea semnătură:

```
int pthread_join(pthread_t thread, void **retval);
```

Parametrul *thread* reprezintă identificatorul thread-ului pe care îl așteptăm, iar *retval* reprezintă valoarea de retur a funcției thread-ului așteptat (și poate fi pus pe NULL dacă nu avem nevoie de această informație). Funcția returnează 0 în caz de succes sau un cod de eroare în caz contrar.

Un exemplu complet de program C care folosește Pthreads se poate observa mai jos. În acest exemplu, se creează două fire de execuție care vor rula aceeași funcție **f**, dar fiecare din ele va primi ca parametru ID-ul curent (0 sau 1). După ce pornește cele două fire de execuție noi, thread-ul principal le așteaptă pe ambele să termine, după care iese și el.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUM_THREADS 2

void *f(void *arg)
{
    long id = *(long*) arg;
    printf("Hello World din thread-ul %ld!\n", id);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int r;
    long id;
    void *status;
    long arguments[NUM_THREADS];

    for (id = 0; id < NUM_THREADS; id++) {
        arguments[id] = id;
        r = pthread_create(&threads[id], NULL, f, (void *) &arguments[id]);

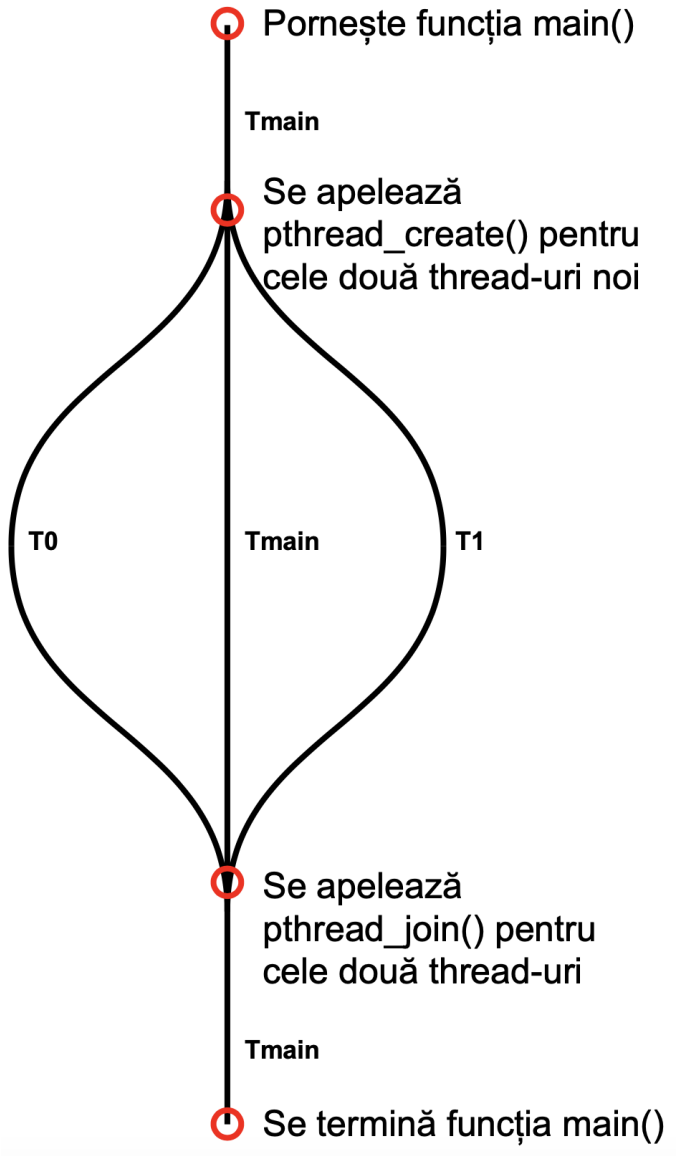
        if (r) {
            printf("Eroare la crearea thread-ului %ld\n", id);
            exit(-1);
        }
    }

    for (id = 0; id < NUM_THREADS; id++) {
        r = pthread_join(threads[id], &status);

        if (r) {
            printf("Eroare la asteptarea thread-ului %ld\n", id);
            exit(-1);
        }
    }

    pthread_exit(NULL);
}
```

O reprezentare vizuală a execuției programului de mai sus poate fi observată în figura de mai jos.



În figura de deasupra, se poate observa că inițial există un singur fir de execuție, cel principal (marcat cu **Tmain**). În momentul în care acesta apelează **pthread_create** de două ori, el creează și pornește alte două fire de execuție noi (**T0** și **T1**) care vor rula funcția **f** cu parametri diferiți (0, respectiv 1). În acest moment, avem trei thread-uri care rulează în paralel. Când thread-ul principal apelează **pthread_join**, acesta se blochează până când cele două fire de execuție secundare se termină (adică până când termină de executat funcția **f**). Dacă unul din thread-urile secundare și-a terminat execuția înainte de apelul **pthread_join**, funcția va returna instant. La finalul programului, thread-ul principal se va opri și el.

Atenție! Găsiți aici un „cheat sheet” cu cele mai importante elemente Pthreads.

Exerciții

1. Compilați și executați codul din fișierul **example.c** din arhiva de laborator. Schimbați numărul de thread-uri și observați cum se schimbă comportamentul programului. **(2p)**
2. Schimbați numărul de thread-uri din cod astfel încât să fie egal cu numărul de core-uri de pe mașina pe care rulați, astfel încât, rulând codul pe un alt calculator, numărul de

thread-uri să se schimbe automat. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(2p)**

3. Modificați funcția **f** astfel încât mesajul „Hello World” să fie afișat iterativ de 100 de ori de fiecare thread, împreună cu indicele iterației. Întrebare: codul afișează mesajele în ordinea așteptată de voi? **(1p)**
4. Modificați programul astfel încât să creeze două thread-uri, fiecare thread rulând propria sa funcție. **(2p)**
5. Pornind de la codul din fișierul **add_serial.c** din arhiva de laborator, paralelizați incrementarea elementelor unui vector cu 100. Acest lucru va presupune împărțirea iterației de adunare la toate thread-urile într-un mod cât mai echitabil. Verificați **Hint 2** de mai jos pentru informații suplimentare. **(2p)**
6. Demonstrați că programul vostru scalează (adică durează mai puțin când rulați cu mai multe thread-uri). Verificați **Hint 3** de mai jos pentru informații suplimentare. **(1p)**

Hint 1

Pentru a putea obține numărul de core-uri de pe un calculator, se poate folosi funcția **sysconf** astfel:

```
#include <unistd.h>

long cores = sysconf(_SC_NPROCESSORS_CONF);
```

Hint 2

Pentru exercițiul 5, avem un vector de **N** elemente pe care vrem să-l împărțim în mod (aproximativ) egal la **P** thread-uri, unde fiecare thread are un **ID** de la 0 la P-1. Astfel, fiecare thread va itera pe câte o secțiune din vectorul inițial, fără a afecta operațiile celorlalte thread-uri. Este necesar deci să calculăm indexul de **start** și indexul de final (**end**) pentru fiecare thread. Un mod de a calcula aceste două valori poate fi următorul:

```
int start = ID * (double)N / P;
int end = min((ID + 1) * (double)N / P, N);
```

Hint 3

Pentru a putea observa mai bine scalabilitatea unui program, este necesar ca acesta să dureze măcar câteva secunde, deoarece, în caz contrar, timpul de inițializare, alte programe care rulează pe calculator și overhead-ul cauzat de planificarea firelor de execuție ar putea afecta timpii de execuție suficient cât să nu vedem scalabilitatea doar prin măsurarea timpului total de execuție.

Mai mult, inițializarea serială a vectorului (în main) are o durată comparabilă cu execuția pe un thread a operației de paralelizat. De aceea, **pentru exercițiul 5, se recomandă să creșteți durata de execuție a unui thread prin repetarea iterativă a operațiilor realizate în funcția de thread**.

Ca să verificați dacă un program scalează, trebuie să îi măsurați durata de execuție atât la o rulare secvențială (cu un singur thread), cât și la o rulare cu mai multe fire de execuție. Pentru acest lucru, puteți folosi comanda **time** în linia de comandă, astfel:

```
$ time ./program
real    0m6.958s
user    0m6.745s
sys     0m0.010s
```

Timpul care ne interesează este cel real (adică timpul „pe ceas”).

Hint 4

O metodă bună de a face debugging la un program C multi-threaded este utiliarul **gdb**. Pe lângă comenzile **gdb** pe care le știți deja, ar mai fi de interes comenzile **info threads** (care afișează informații despre thread-urile existente la momentul curent de timp) și **thread <N>** (care mută contextul de execuție pe thread-ul N).

Laboratorul 2 - Elemente de sincronizare în Pthreads

Responsabili: Radu Ciobanu, Andrei Damian, Delia Stuparu, Dragoș Cocîrlea

Introducere

Când vorbim de programarea paralelă, putem avea situații în care mai multe fire de execuție care rulează în paralel vor să acceseze simultan aceleași resurse. De exemplu, putem avea situația următoare. Avem două thread-uri (**T0** și **T1**) care au acces partajat la o variabilă întreagă *a* inițializată cu 0. În funcția de thread, atât **T0** cât și **T1** incrementează *a* cu 2. În mod normal, ne-am aștepta ca valoarea variabilei *a* după execuția programului nostru să fie 4, pentru că avem două incrementări ale variabilei în cele două thread-uri.

În realitate, situația nu este întotdeauna așa. Dacă am traduce incrementarea unei variabile întregi în cod de asamblare, această operație ar putea arăta în felul următor (în exemplul de mai jos, *eax0* reprezintă registrul *eax* al thread-ului **T0**, iar *eax1* desemnează registrul *eax* al thread-ului **T1**):

T0	T1
load(a, eax0)	load(a, eax1)
eax0 = eax0 + 2	eax1 = eax1 + 2
write(a, eax0)	write(a, eax1)

Putem avea următorul scenariu:

- **T0** citește valoarea lui *a* (0) în propriul registru *eax0*
- în același timp, **T1** citește valoarea lui *a* (tot 0) în propriul registru *eax1*
- **T0** incrementează valoarea lui *eax0*, care devine 2
- **T1** face același lucru, iar *eax1* devine tot 2
- **T0** scrie valoarea din *eax0* în a, care devine 2
- **T1** scrie valoarea din *eax1* în a, care rămâne tot 2.

Se poate deci observa că, în funcție de modul în care thread-urile **T0** și **T1** sunt planificate, este posibil ca rezultatul secvenței de mai sus să fie 2 sau 4. Acest lucru se numește **race condition** și este cauzat de faptul că rezultatul calculului este condiționat de modul de planificare a unor evenimente necontrolabile. Operația de incrementare a lui *a* cu 2 nu este **atomică**, fiind compusă din mai multe operații care se pot intercala atunci când rulăm pe mai multe fire de execuție.

Mutex

Un **mutex** (abreviere de la „mutual exclusion”) este o primitivă de sincronizare prin care putem proteja accesul la date atunci când avem scrieri (potențial) concurente. El funcționează ca un „zăvor” ce protejează accesarea unor resurse partajate.

Un mutex se folosește pentru a delimita o **regiune critică**, adică o zonă a programului în care se poate afla cel mult un thread la un moment dat de timp. Dacă un thread **T1** încearcă să intre într-o regiune critică atunci când alt thread **T0** este deja acolo, **T1** se va bloca până când **T0** va ieși din regiunea critică.

În cadrul exemplului de mai sus, am putea folosi un mutex pentru a defini o regiune critică în jurul operației de incrementare a lui *a*, lucru care ar face imposibilă intercalarea operațiilor celor două thread-uri. Primul thread care intră în regiunea critică va incrementa *a* în mod exclusiv la 2, iar cel de-al doilea thread nu va putea incrementa *a* decât atunci când el este deja 2.

Un mutex are două operații principale: închidere (**lock**) și deschidere (**unlock**). Prin închidere, un thread marchează intrarea în zona critică, adică specifică faptul că orice alt thread care va încerca să facă o operație de închidere va trebui să aștepte. Prin deschidere, se marchează ieșirea din zona critică și deci permisiunea ca un alt thread să intre în zona critică.

În Pthreads, o secvență tipică de folosire a unui mutex arată în felul următor:

1. se creează și se inițializează o variabilă de tip mutex
2. mai multe thread-uri încearcă să închidă mutexul (adică să intre în zona critică)
3. unul singur dintre ele reușește acest lucru și ajunge să „dețină” mutexul (adică se află în regiunea critică)
4. thread-ul aflat în zona critică realizează diverse operații pe datele protejate
5. thread-ul care deține mutexul iese din zona critică (deschide mutexul)
6. alt thread intră în zona critică și repetă procesul
7. la final, variabila de tip mutex este distrusă.

În Pthreads, un mutex se reprezintă printr-o variabilă de tip *pthread_mutex_t*, și se inițializează folosind următoarea funcție:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Primul parametru reprezintă o referință la variabila mutex, iar al doilea parametru specifică atributele mutexului nou-creat (dacă se dorește un comportament implicit, parametrul *attr* se poate lăsa NULL).

Pentru a dezaloca un mutex, se folosește următoarea funcție, care primește ca parametru un pointer la mutexul care urmează a fi dezalocat:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Pentru a se face lock pe un mutex, se folosește următoarea funcție, care primește ca parametrul mutexul:

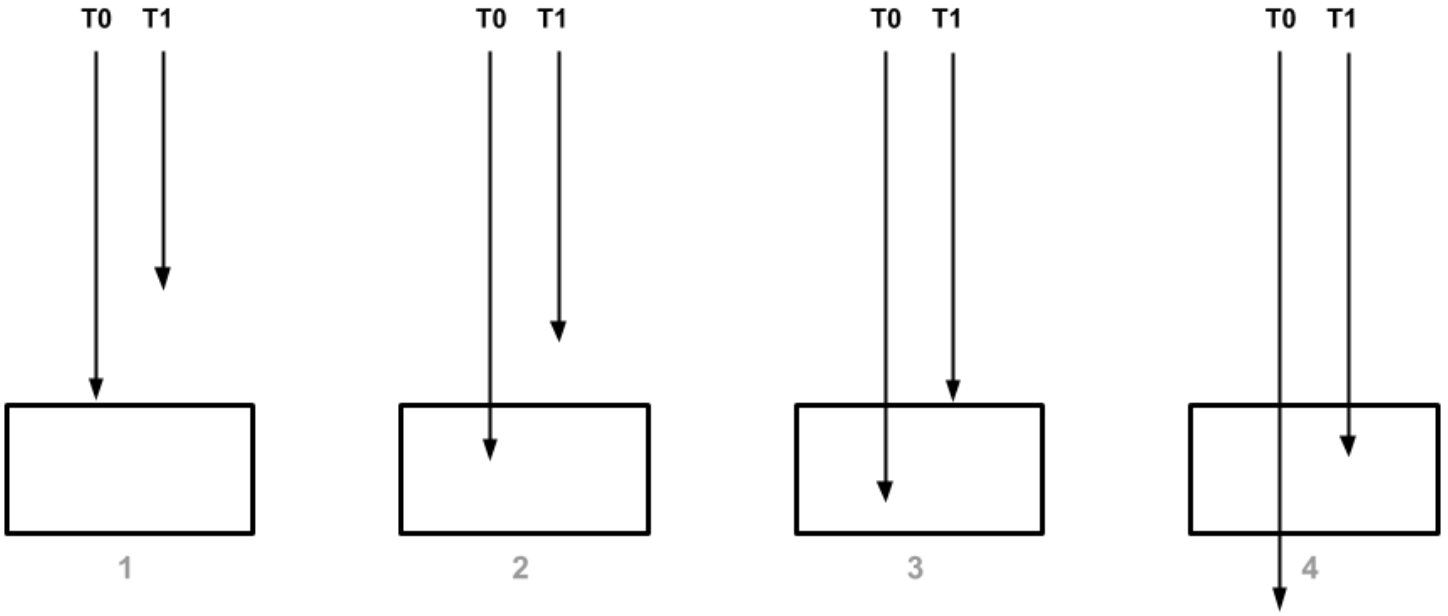
```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

Operația inversă, prin care se specifică ieșirea dintr-o zonă critică (adică deschiderea mutexului), este executată prin intermediul următoarei funcții:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Toate patru funcțiile de mutex returnează 0 dacă s-au executat cu succes, sau un cod de eroare în caz contrar.

O reprezentare grafică a funcționării unui mutex se poate vedea în figura de mai jos, într-un scenariu în care avem două thread-uri (**T0** și **T1**) și o regiune critică controlată de mutex (chenarul negru din imagine). La momentul de timp **1** (în partea stângă a figurii), **T0** încearcă să intre în regiunea critică. Pentru că, la momentul respectiv de timp, niciun alt thread nu deține mutexul (adică nu se află în regiunea critică), **T0** intră în regiunea critică (momentul de timp **2**). Mai departe, atunci când **T1** ajunge la intrarea în regiunea critică (încearcă să facă lock pe mutex) la momentul de timp **3**, se blochează pentru că mutexul este curent deținut de **T0** (acesta se află în regiunea critică). Abia în momentul în care **T0** a ieșit din zona critică (la momentul de timp **4**), **T1** se va putea debloca și își va continua execuția.



Atenție! Dacă vrem să protejăm o secțiune din programul nostru folosind un mutex, atunci fiecare thread care accesează acea secțiune trebuie să facă lock și unlock pe aceeași variabilă mutex. De asemenea, dacă un thread vrea să facă unlock pe un mutex pe care nu îl deține (nu a făcut lock pe el în prealabil), va rezulta un comportament nedefinit.

Barieră

O altă primitivă de sincronizare folosită în calculul paralel este **bariera**. Ea are rolul de a se asigura că niciun thread nu poate trece mai departe de punctul în care este plasată decât atunci când toate thread-urile gestionate de barieră ajung în acel punct. Un exemplu de utilizare este atunci când împărțim un calcul pe mai multe thread-uri și vrem să nu mergem mai departe cu execuția programului decât în momentul în care fiecare thread și-a terminat propriile calcule.

În Pthreads, o barieră este reprezentată prin tipul *pthread_barrier_t* și inițializată prin următoarea funcție:

```
int pthread_barrier_init(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count);
```

Primul parametru reprezintă o referință la barieră, al doilea parametru poate fi folosit pentru setarea unor atribute ale barierei (la fel ca la mutex), iar ultimul parametru denotă numărul de thread-uri care trebuie să ajungă la barieră pentru ca aceasta să se deblocheze. Acest lucru înseamnă că bariera are un contor intern care numără thread-urile care așteaptă deblocarea ei. Atunci când contorul ajunge la numărul setat la inițializarea barierei, thread-urile își pot continua execuția paralelă.

Pentru a dezaloca o barieră, se folosește următoarea funcție:

```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Ambele funcții returnează 0 dacă s-au executat cu succes sau un cod de eroare în caz contrar.

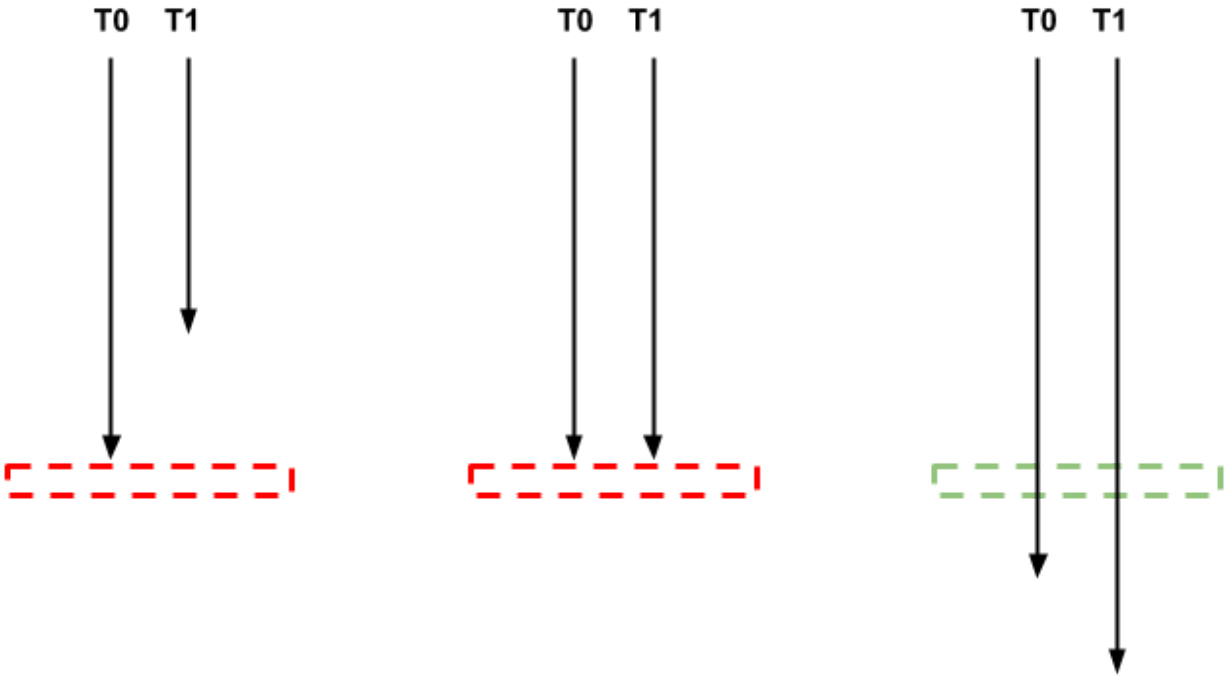
Pentru a face un thread să aștepte la o barieră (pentru a „pune o barieră” în cod), se folosește următoarea funcție:

```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

Funcția de mai sus va returna PTHREAD_BARRIER_SERIAL_THREAD pentru un singur thread arbitrar de la barieră și 0 pentru toate celelalte. Dacă funcția a avut vreo eroare, un cod eroare va fi returnat.

Atenție! Fiecare thread care trebuie să aștepte la barieră va apela funcția de mai sus pe aceeași variabilă de tip *pthread_barrier_t*. Dacă numărul de thread-uri care apelează **pthread_barrier_wait** este mai mic decât parametrul cu care a fost inițializată bariera, aceasta nu se va debloca niciodată.

O reprezentare grafică a modului de funcționare a unei bariere se poate vedea în figura de mai jos, unde avem o barieră inițializată cu 2. Atunci când thread-ul **T0** ajunge la barieră, acesta se blochează în așteptare, așa cum se observă în partea stângă a imaginii. La un moment dat de timp, **T1** va ajunge și el la barieră, așa cum se vede în centrul imaginii. Abia în acel moment, cele două thread-uri își vor putea continua execuția individual, cum se observă în partea dreapta a imaginii.



Mai multe informații detaliate despre API-ul Pthreads găsiți în tutorialul LLNL [https://computing.llnl.gov/tutorials/pthreads/] și în man [https://man7.org/linux/man-pages/man7/pthreads.7.html].

Exerciții

1. Compilați fișierul **mutex.c** din arhiva de laborator și rulați de mai multe ori programul obținut. Veți observa că rezultatul nu este întotdeauna cel așteptat. Rezolvați problema de sincronizare folosind un mutex. **(2p)**
2. Pornind de la fișierul **barrier.c** din arhiva de laborator, folosiți o barieră pentru a vă asigura că output-ul va fi întotdeauna „1\n2”. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(2p)**
3. Pornind de la fișierul **multiply.c** din arhiva de laborator, paralelizați programul prin împărțirea iterației exterioare la mai multe thread-uri. Verificați corectitudinea și scalabilitatea programului rezultat. Verificați **Hint 2** de mai jos pentru informații suplimentare. **(2p)**
4. Pornind de același cod, paralelizați doar cea de-a doua buclă de iterație. Verificați corectitudinea și scalabilitatea programului rezultat. **(2p)**
5. Pornind de la același cod, paralelizați doar bucla interioară. Verificați corectitudinea și scalabilitatea programului rezultat. Verificați **Hint 3** de mai jos pentru informații suplimentare. **(2p)**
6. Pornind de la fișierul **strassen.c** din arhiva de laborator, paralelizați înmulțirea unor matrice cu algoritmul Strassen folosind 7 thread-uri. Verificați **Hint 4** de mai jos pentru informații

suplimentare. **(1p bonus)**

Hint 1

Pe sisteme MacOS, biblioteca de Pthreads nu conține implementarea pentru barieră. Pentru a putea face totuși acest exercițiu, există în arhiva de laborator un fișier numit *pthread_barrier_mac.h* pe care trebuie să-l includeți în fișierul vostru sursă.

Hint 2

Pentru a putea testa corectitudinea paralelizării de la exercițiile 3, 4 și 5, găsiți în arhiva de laborator un script numit *test.sh*. Acesta face următorii pași:

1. verifică să existe un binar numit *multiply_seq* pentru implementarea secvențială a înmulțirii de matrice, pentru care aveți fișierul sursă *multiply_seq.c* în arhiva de laborator (acesta va servi drept etalon pentru corectitudinea implementării paralele)
2. verifică să existe un binar numit *multiply* pentru implementarea voastră paralelă
3. rulează programul secvențial
4. rulează programul paralel
5. compară rezultatele celor două rulări cu *diff*; dacă nu există diferențe, scriptul nu afișează nimic; dacă implementarea paralelă este incorectă, se vor afișa diferențele dintre rularea implementării secvențiale și a celei paralele.

În mod implicit, scriptul rulează pe matrice de 1000×1000 de elemente, cu două thread-uri pentru implementarea paralelă. Dacă doriți să modificați aceste valori (și vă recomandăm să o faceți, pentru o testare cât mai completă), puteți modifica valorile variabilelor **N** și **P** din script.

Hint 3

Veți observa la acest exercițiu că, dacă paralelizați bucla interioară așa cum ați făcut la precedentele două exerciții, rezultatele nu vor fi întotdeauna corecte. De ce? Ce trebuie să faceți pentru o implementare corectă?

Hint 4

Algoritmul lui Strassen [http://stanford.edu/~rezab/classes/cme323/S16/notes/Lecture03/cme323_lec3.pdf] este un algoritm pentru înmulțire de matrice mai rapid decât metoda standard, având o complexitate de $O(N^{2.8074})$. În acest algoritm, la primul pas se definesc 7 matrice adiționale obținute prin înmulțiri ale matricelor bloc obținute din matricele inițiale. La al doilea pas, aceste 7 matrice noi sunt folosite pentru a calcula (prin operații de adunare și scădere) componentele bloc ale matricei finale.

Pentru exercițiul 6, aveți deja implementat calculul matricelor adiționale și calculul final, așa că voi trebuie doar să paralelizați aceste operații.

Laboratorul 3 - Algoritmi paraleli de sortare

Responsabili: Radu Ciobanu, Andrei Damian, Delia Stuparu, Dragoș Cocîrlea

În acest laborator, vom discuta despre câteva exemple de algoritmi paraleli de sortare.

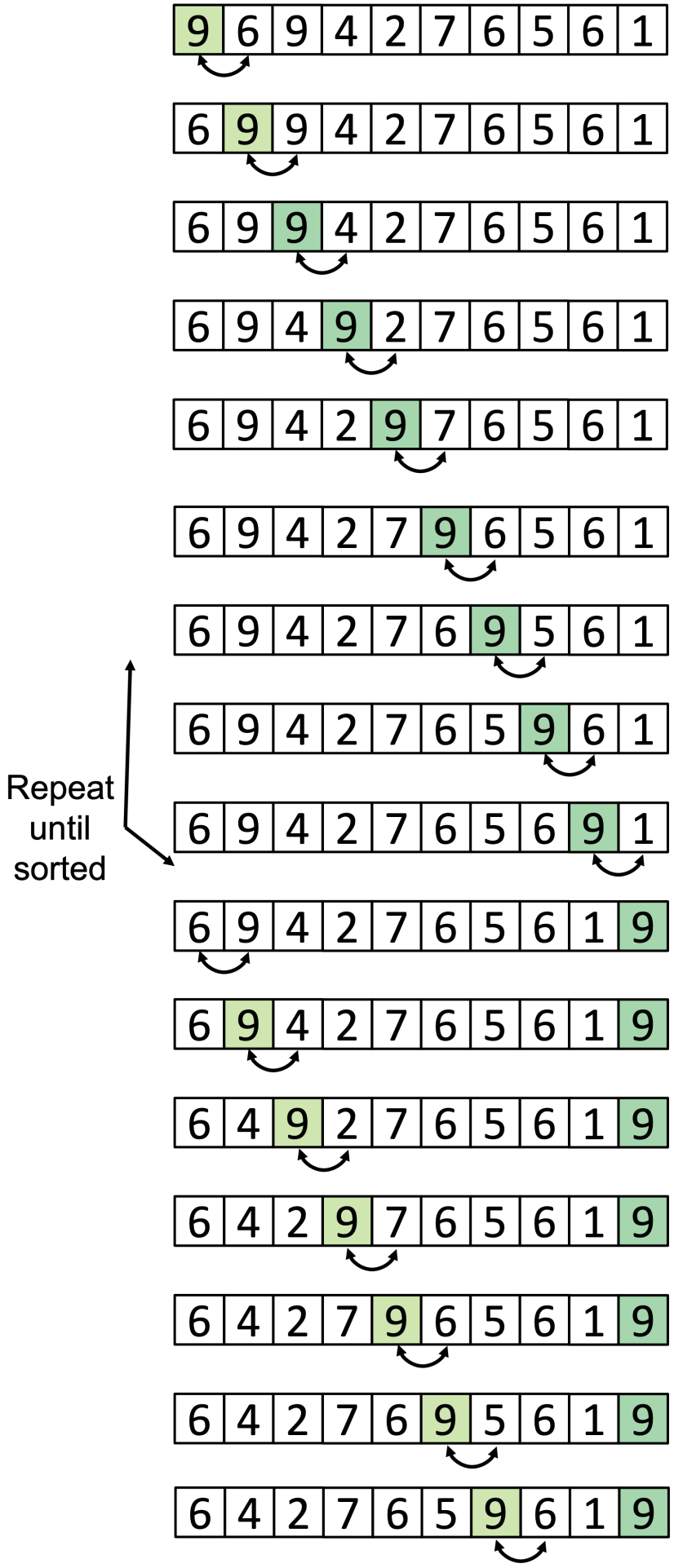
Odd-even transposition sort (OETS)

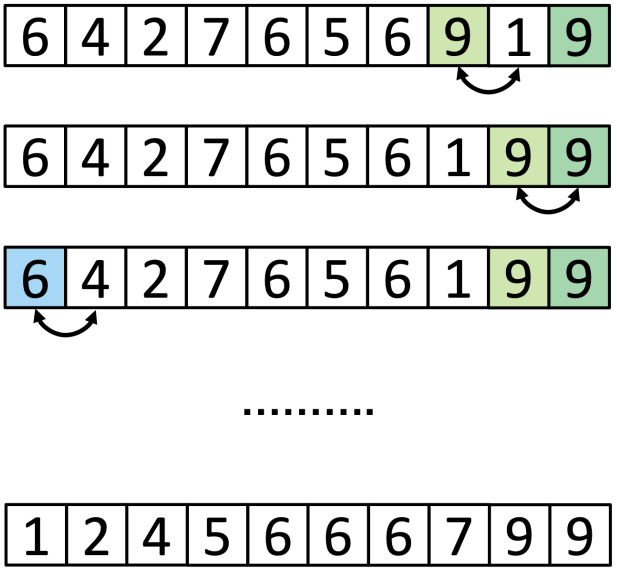
Unul din cei mai cunoscuți (chiar dacă nu neapărat eficienți) algoritmi de sortare este **bubble sort**. Acesta funcționează pe baza următorului pseudocod:

```
function bubbleSort(list) {
  sorted = false;
  while (!sorted) {
    sorted = true;
    for (var i = 0; i < list.length - 1; i++) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
        sorted = false;
      }
    }
  }
}
```

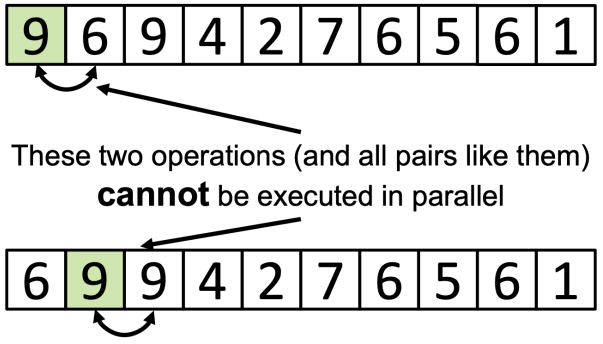
Așa cum se poate observa mai sus, bubble sort parcurge șirul de sortat element cu element, comparând elementul curent cu vecinul din dreapta. Dacă numărul din dreapta este mai mic, se realizează o interschimbare între elementul curent și cel din dreapta sa. Complexitatea acestui algoritm este $\mathbf{O(N^2)}$, pentru că se termină în cel mult N parcurgeri ale șirului (unde N este numărul de elemente din șirul de sortat).

Un exemplu de comportament al bubble sort se poate observa în imaginea de mai jos.


9 6 9 4 2 7 6 5 6 1
6 9 9 4 2 7 6 5 6 1
6 9 9 4 2 7 6 5 6 1
6 9 4 9 2 7 6 5 6 1
6 9 4 2 9 7 6 5 6 1
6 9 4 2 7 9 6 5 6 1
6 9 4 2 7 6 9 5 6 1
6 9 4 2 7 6 5 9 6 1
6 9 4 2 7 6 5 6 9 1
6 9 4 2 7 6 5 6 1 9
6 9 4 2 7 6 5 6 1 9
6 4 9 2 7 6 5 6 1 9
6 4 2 9 7 6 5 6 1 9
6 4 2 7 9 6 5 6 1 9
6 4 2 7 6 9 5 6 1 9
6 4 2 7 6 5 9 6 1 9
Repeat until sorted



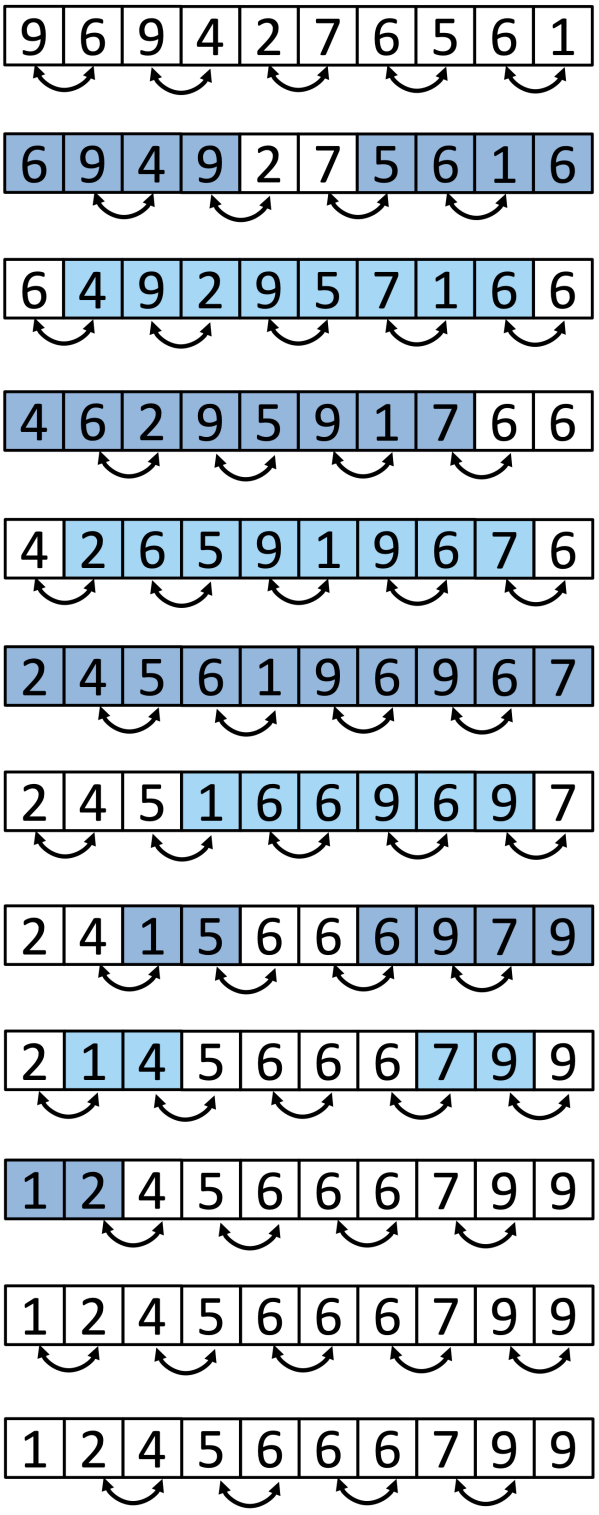
Dacă am dori să paralelizăm acest algoritm, un potențial mod de abordare ar fi să realizăm în paralel comparația și potențiala interschimbare de elemente vecine, dar acest lucru ar putea duce la problema prezentată în imaginea de mai jos.



Mai precis, operații pe elemente adiacente nu se pot realiza simultan, pentru că se poate ajunge la un race condition. Din acest motiv, un mod de a paraleliza bubble sort este dat de algoritmul **odd-even transposition sort**, care funcționează după următorul pseudocod:

```
function oddEvenSort(list) {
  for (var k = 0; k < list.length; k++) {
    for (i = 0; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
      }
    }
    for (i = 1; i < list.length - 1; i += 2) {
      if (list[i] > list[i + 1]) {
        swap(list[i], list[i + 1]);
      }
    }
  }
}
```

Așa cum se poate observa mai sus, odd-even transposition sort are două faze. În **faza pară**, elementele de pe poziții pare din șirul de sortat sunt comparate (și eventual interschimbate) cu vecinii din dreapta. După ce se termină faza pară (adică după ce toate elementele pare au fost procesate), urmează **faza impară**, în care elementele impare sunt analizate și comparate cu vecinii din dreapta. La fel ca la bubble sort, numărul maxim de iterații necesare pentru a sorta un șir va fi N (numărul de elemente din șir). Dacă avem P fire de execuție, complexitatea acestui algoritm va fi **O(N/P*N)**, sau **O(N)** pentru P=N. În imaginea de mai jos, se poate observa o reprezentare grafică a modului de funcționare al odd-even transposition sort.



Algoritmul odd-even transposition sort a fost gândit inițial pentru a fi rulat pe șiruri de procesoare (processor arrays [https://en.wikipedia.org/wiki/Massively_parallel_processor_array]), unde un procesor conține o singură valoare din șirul de sortat și poate comunica doar cu procesorul din stânga și cu cel din dreapta.

Shear sort

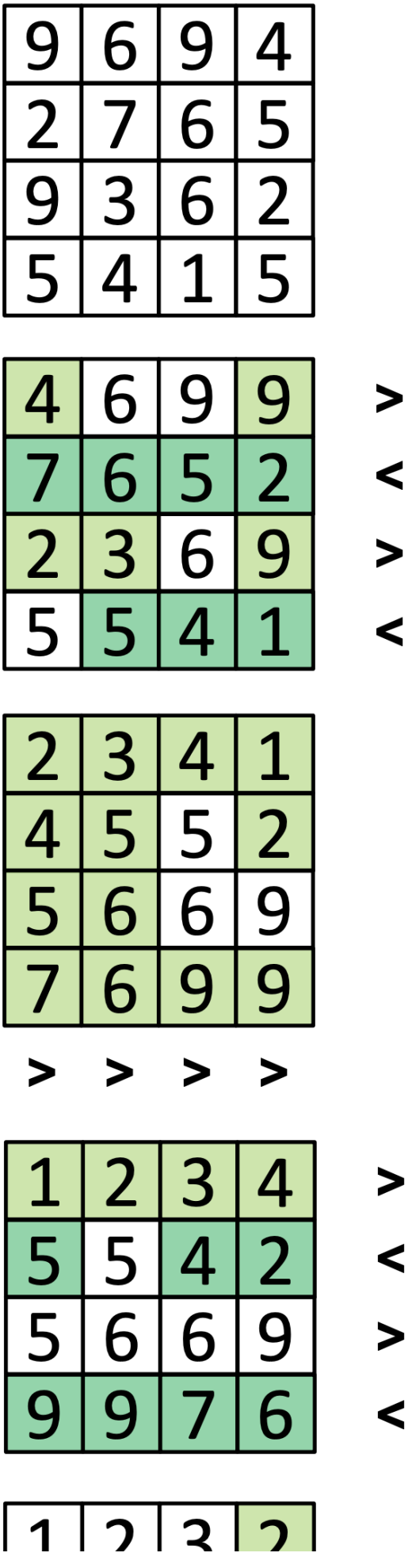
Un alt exemplu de algoritm de sortare care a fost conceput pentru sisteme multi-procesor unde un procesor este conectat doar la o parte din celelalte procesoare este **shear sort** (cunoscut de asemenea ca **row-column sort** sau **snake-order sort**), care presupune că lucrăm pe procesoare conectate într-o formă de matrice. Astfel, un procesor poate să comunice cu vecinii din stânga, din dreapta, de sus și de jos. Dacă ne imaginăm deci că procesoarele sunt așezate într-o matrice, cele două faze ale algoritmului shear sort sunt următoarele:

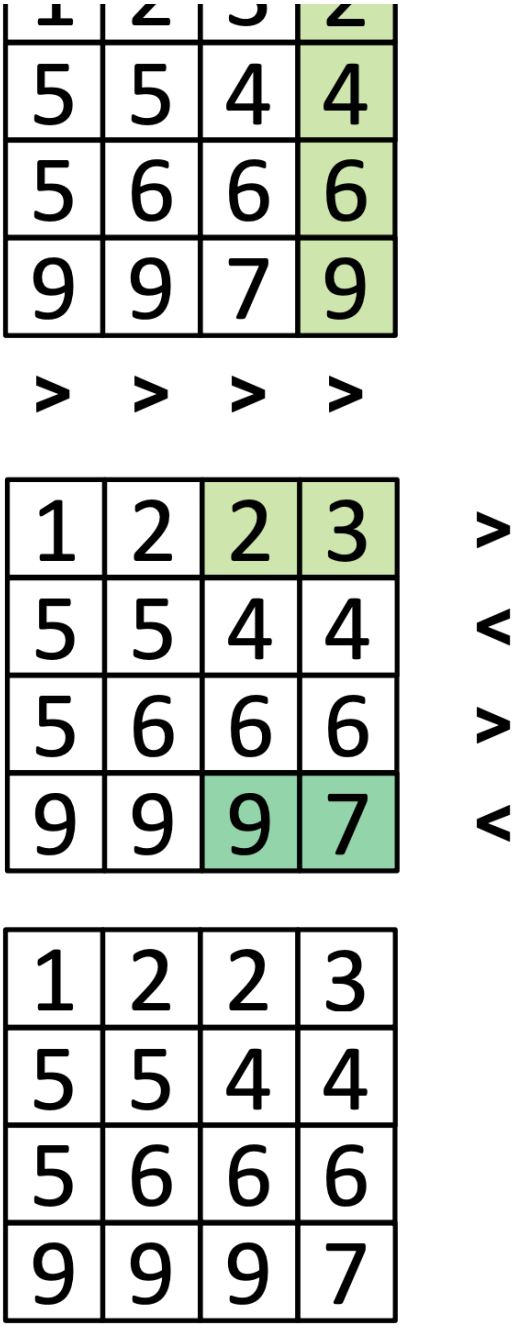
- se sortează liniile matricei astfel încât randurile pare au valorile ordonate crescător, iar rândurile impare au valorile ordonate descrescător
- se sortează coloanele crescător.

Se garantează că algoritmul va sorta numerele după cel mult **sup($\log_2 N$) + 1** faze, unde N este numărul de elemente ce trebuie sortate. Din acest motiv, algoritmul are complexitatea **O($N \log_2 N$)**. Pseudocodul algoritmului este prezentat mai jos.

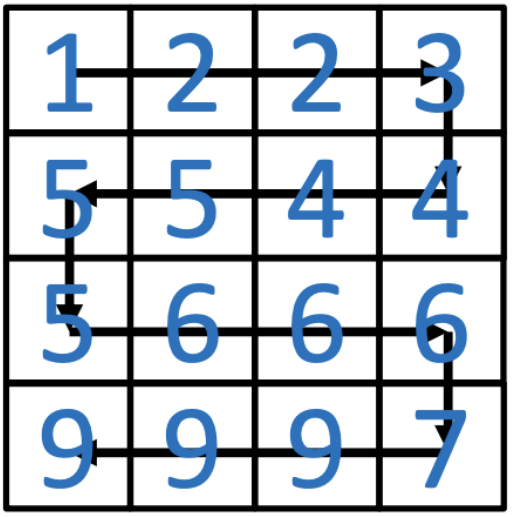
```
function shearSort(matrix) {
  for (k = 0; k < ceil(log2(matrix.lines * matrix.columns)) + 1; k += 2) {
    for (i = 0; i < matrix.lines; i += 2) {
      sortAscendingLine(i);
    }
    for (i = 1; i < matrix.lines; i += 2) {
      sortDescendingLine(i);
    }
    for (var i = 0; i < matrix.columns; i++) {
      sortAscendingColumn(i);
    }
  }
}
```

O reprezentare grafică a funcționării shear sort este prezentată în figura de mai jos.


9 6 9 4
2 7 6 5
9 3 6 2
5 4 1 5
4 6 9 9 >
7 6 5 2 <
2 3 6 9 >
5 5 4 1 <
2 3 4 1
4 5 5 2
5 6 6 9
7 6 9 9
> > > >
1 2 3 4 >
5 5 4 2 <
5 6 6 9 >
9 9 7 6 <
1 2 3 2



La finalul rulării algoritmului, lista de numere va fi sortată într-un mod „șerpuit”, de unde și numele algoritmului. Acest lucru se poate observa în imaginea de mai jos.



Merge sort paralel

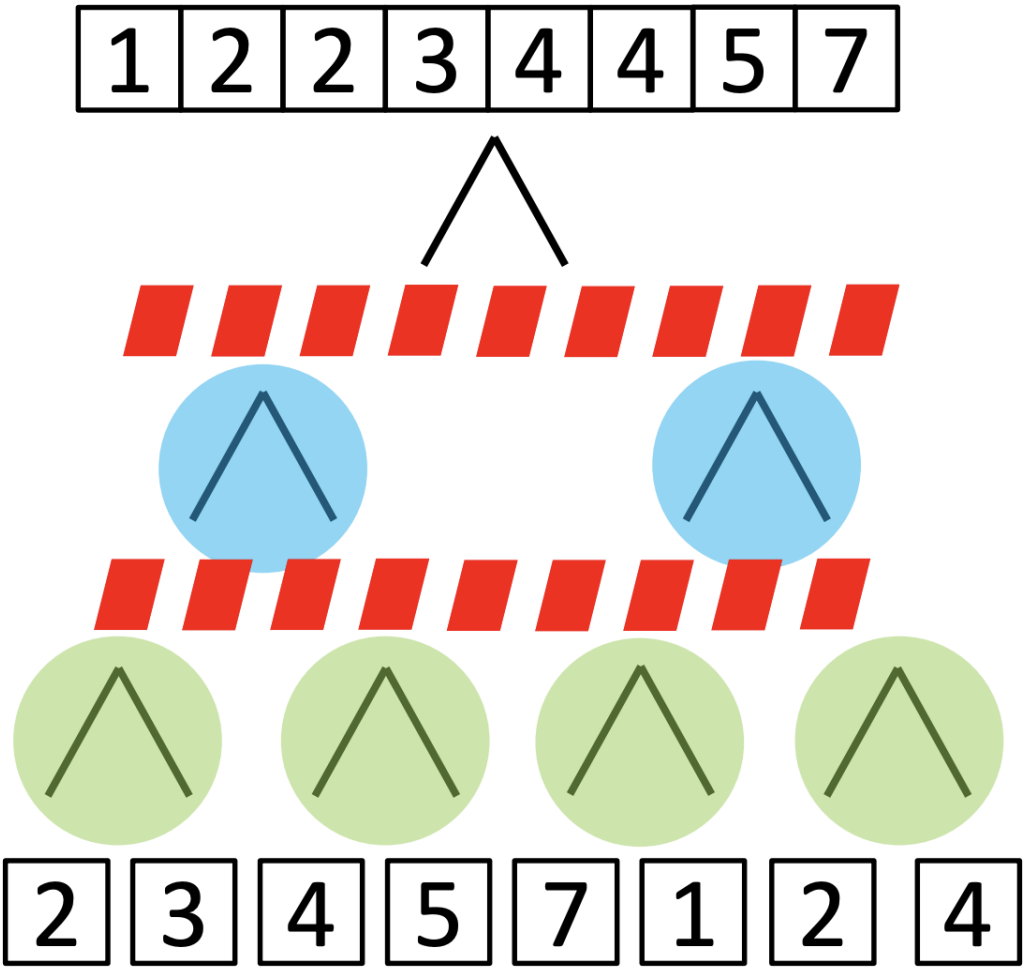
Merge sort (sau sortarea prin interclasare) este un algoritm de sortare de tip ***divide et impera*** care presupune următorii pași generali:

1. se împarte șirul de N elemente de sortat în N șiruri de lungime 1
2. se aplica operația de interclasare („merge”) între câte două astfel de șiruri de lungime 1, rezultând N/2 șiruri sortate de lungime 2
3. se repetă pașii de mai sus realizând interclasări între șiruri din ce în ce mai mari, până se ajunge la un șir sortat de N elemente.

Numărul de pași de interclasare necesari este $\log_2 N$, iar operațiile de interclasare de la un pas se realizează în O(N), deci complexitatea algoritmului merge sort este **$O(N\log_2 N)$**.

Pentru a paraleliza acest algoritm, putem observa că operațiile de interclasare de la fiecare pas se pot realiza în paralel. Totuși, operațiile de „merge” de la fiecare pas trebuie terminate în totalitate înainte de a trece la următorul pas, deci avem nevoie de o barieră (sau un mecanism similar) după fiecare pas de interclasare. Se poate observa că gradul de paralelism de la un pas de interclasări este din ce în ce mai mic pe măsură ce avansăm în algoritm, pentru că numărul de operații de „merge” de la fiecare pas scade. Complexitatea algoritmului paralel este **O(N)** pentru P=N.

O reprezentare grafică a algoritmului de merge sort paralel se poate observa în imaginea de mai jos, unde operațiile cu aceeași culoare pot fi realizate în paralel, iar simbolurile cu roșu reprezintă bariere.



Exerciții

1. Pornind de la implementarea secvențială a algoritmului bubble sort aflată în fișierul **oets.c** din arhiva de laborator, implementați și paralelizați algoritmul odd-event transposition sort. **(3p)**
2. Analizați scalabilitatea implementării de la punctul precedent. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(1p)**
3. Pornind de la implementarea secvențială a algoritmului shear sort aflată în fișierul **shear.c** din arhiva de laborator, paralelizați algoritmul. **(3p)**
4. Analizați scalabilitatea implementării de la punctul precedent. Verificați **Hint 1** de mai jos pentru informații suplimentare. **(1p)**
5. Pornind de la implementarea secvențială a algoritmului merge sort aflată în fișierul **merge.c** din arhiva de laborator, paralelizați algoritmul. Verificați **Hint 2** de mai jos pentru informații suplimentare. **(2p)**

Hint 1

În toate fișierele sursă din arhiva de laborator, verificarea corectitudinii implementării paralele se realizează prin sortarea șirului folosind quick sort într-un array folosit ca etalon la final (se realizează o comparație element cu element). Atunci când doriți să verificați scalabilitatea programului vostru, comentați sau eliminați sortarea folosind quick sort și comparația aferentă, pentru a avea niște timpi cât mai corecți. De asemenea, încercați să eliminați toate afișările în terminal.

Hint 2

Implementarea merge sort din arhiva de laborator funcționează pentru un N putere a lui 2. Testați-vă implementarea paralelă doar pe acest caz.

Laboratorul 4 - Introducere în Java Multithreading

Responsabili: Alexandru-Ionuț Mustață, Gabriel Guțu-Robu

Suportul de multithreading oferit de Java

Spre deosebire de limbajul C unde implementarea unui thread și a mecanismelor de sincronizare depinde de o bibliotecă ce este specifică unui anumit tip de sistem de operare (Linux, Windows, MacOSX), Java oferă suport pentru lucrul cu thread-urile direct din cadrul SDK său.

Implementarea unui nou thread

În Java există două modalități prin care putem să implementăm un nou thread.

Prima variantă de implementare are la bază crearea unei clase ce implementează interfața **Runnable** ce conține metoda **void run()**. Codul ce va reprezenta logica noului thread va fi plasat în interiorul acestei metode.

```
public class Task implements Runnable {
    public void run() {
        System.out.println("Hello from my new thread!");
    }
}
```

O altă metodă de implementare a unui thread constă din crearea unei clase ce extinde clasa **Thread** și suprascrie (overrides) metoda **void run()** din cadrul acesteia. La fel ca în primul caz, logica noului thread va fi implementată în cadrul acestei metode.

```
public class MyThread extends Thread {
    public void run() {
        System.out.println("Hello from my new thread!");
    }
}
```

Rularea în paralel a noului thread

În cazul în care s-a folosit mecanismul de implementare a interfeței **Runnable**, atunci pentru a putea crea un thread nou ce va conține logica definită în clasa **Task** se va crea o instanță a clasei **Task** ce va fi furnizată ca parametru constructorului clasei **Thread**. Pentru a rula în paralel noul thread creat cu ajutorul constructorului se va apela metoda **public void start()** a acestuia.

```
public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new Task());
        t.start();
    }
}
```

În cazul în care s-a extins clasa **Thread** pentru implementarea noului tip de thread se va putea crea un thread nou prin instanțierea directă a clasei **MyThread** și pentru a se porni excecuția în paralel a acestuia se va apela metoda **public void start()** moștenită din cadrul clasei **Thread**.

```
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Atenție! Există o distincție foarte importantă între metodele de **start()** și **run()** ale clasei **Thread**! Atunci când este apelată metoda **run()** codul prezent în aceasta se va executa secvențial în cadrul thread-ului care a apelat-o. Atunci când este apelată metoda **start()** JVM (Java Virtual Machine) va crea un nou thread ce va executa instrucțiunile prezente în cadrul metodei **run()** în paralel cu thread-ul care a apelat metoda **start()**.

Așteptarea terminării execuției unui thread

Pentru a aștepta terminarea execuției unui thread, Java ne pune la dispoziție metoda **public final void join()** a clasei **Thread**. Trebuie ținut cont de faptul că această metodă poate arunca excepții de tipul **InterruptedException**.

```
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();

        try {
            t.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Trimiterea de parametrii unui thread și obținerea de rezultate de la acesta

Pentru a trimite parametrii unui thread se va folosi constructorul clasei care inglobeaza logica thread-ului pe care dorim sa il implementam (indiferent de metoda de implementare a acestuia - prin moștenire sau implementeare de interfață). Pentru a obține un rezultat de la un thread care și-a terminat execuția (apelul metodei **join()** a întors) putem folosi fie metode de tip getter care vor întoarce rezultatul fie putem accesa în mod direct câmpul rezultat în cazul în care acest este definit ca public.

```
public class Task extends Thread {

    private int id;
    private int result;

    public Task(int id) {
        this.id = id;
    }

    public void run() {
        result = id * id;
    }

    public int getResult() {
        return result;
    }

}
```

```
public class Main {

    public static void main(String[] args) {
        int NUMBER_OF_THREADS = 4;
        Thread[] t = new Thread[NUMBER_OF_THREADS];

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            t[i] = new Task(id);
            t[i].start();
        }

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            try {
                t[i].join();
                System.out.println("Thread " + i + " computed result " + ((Task)t[i]).getResult() + ".");
            } catch (InterruptedException e) {
```

```
                e.printStackTrace();
            }
        }
    }
}
```

Synchronized

Cuvântul rezervat **synchronized** are rolul de a defini blocuri de cod și metode ce reprezintă secțiuni/regiuni critice.

```
public class MyConcurrentArray<T> {

    private static int numberOfInstances = 0;
    private T[] content;

    public MyConcurrentArray(int size) {
        if (size > 0) {
            content = new T[size];
        } else {
          throw new RuntimeException("Negative size provided for MyConcurrentArray instantiation.");
        }

        synchronized(MyConcurrentArray.class) {
            ++numberOfInstances;
        }
    }

    //Metodă sincronizată.
    public synchronized T get(int index) {
        if (index < content.length) {
            return content[index];
        }
        throw new IndexOutOfBoundsException(index + " is out of bounds for MyConcurrentArray of size " + content.length);
    }

    public void set(int index, T newT) {
        //Bloc de cod sincronizat ce folosește instanța curentă (this) pe post de zăvor.
        synchronized(this) {
            if (index < content.length) {
                content[index] = newT;
            }
            throw new IndexOutOfBoundsException(index + " is out of bounds for MyConcurrentArray of size " + content.length);
        }
    }

    //Metodă sincronizată statică.
    public static synchronized int getNumberOfInstances(){
        return numberOfInstances;
    }

    public void size() {
        return content.length;
    }
}
```

Pentru a putea discuta comportamentul blocurilor de cod sincronizate vom folosi exemplul de mai sus în care este definită o clasă ce implementează sumar conceptul de vector de dimensiune fixă ce poate fi folosit într-un program multithreading (structură de date thread-safe).

Se observă faptul ca metoda **get** este definită ca fiind **synchronized**. În momentul când un thread va apela aceasta metodă pe o instanță a clasei **MyConcurrentArray** va trebui mai întâi să obțină monitorul [https://en.wikipedia.org/wiki/Monitor_%28synchronization%29] asociat acesteia (obiectului) pentru a putea executa corpul metodei. Dacă monitorul nu este deținut de nici un alt thread, atunci thread-ul apelant va putea să execute corpul de instrucțiuni al metodei, altfel se va bloca (va aștepta) până când acesta devine disponibil. La încheierea execuției corpului metodei acesta restituie accesul la monitor.

În cazul metodei **set** este prezent un bloc de instrucțiuni sincronizat. Acesta va folosi monitorul obiectului desemnat între paranteze pentru a oferi thread-ului curent acces excluziv în cadrul regiunii critice. În cazul

nostru se folosește instanța curentă a obiectului (**this**). Mecanismul de intrare și ieșire în/din secțiunea critică este același ca cel prezentat mai sus pentru cazul metodelor sincronizate.

În cazul apelării metodelor sincronizate statice se va încerca obținerea monitorului asociat clasei pentru a executa codul acestora. Acest lucru se întâmplă deoarce o metodă statică aparține clasei, ea nu aparține nici unei instanțe a clasei. De aceea când este necesar accesul exclusiv la un câmp static dintr-o clasă se va folosi clasa în antetul blocului synchronized (MyClass.class, după cum este prezentat și în constructorul din exemplu).

Atenție! Metodele și blocurile de cod **synchronized** din Java sunt **reentrante**. Dacă un thread a obținut monitorul unui obiect, atunci el va putea intra în orice alt bloc și metodă sincronizate ce sunt asociate cu acel obiect (implicit cu acel monitor). Acest comportament nu este activat în mod implicit pentru **pthread_mutex_t** definit în C (se poate obține prin specificarea de atribute la creare, **Hint**: PTHREAD_MUTEX_RECURSIVE).

CyclicBarrier

Bariera ciclică reprezintă un mecanism de (re)sincronizare al mai multor thread-uri care are scopul de a bloca un număr specificat de thread-uri și de a le lăsa sa își continua execuția într-un mod sincron după ce toate au apelat metoda acesteia de resincronizare. În Java aceast mecanism este reprezentat de clasa CyclicBarrier. La instanțierea acesteia se va specifica numărul de thread-uri pe care aceasta le va resincroniza. Acest mecanism de sincronizare al thread-urilor este util în cazul algoritmilor iterativi ce sunt rulați în paralel și au nevoie de o etapă de resincronizare a firelor de execuția înainte de a trece la noua iterație de calcul. Trebuie ținut cont de faptul că apelul metodei **await()** pe bariera ciclică poate arunca o excepție de tipul **BrokenBarrierException** sau **InterruptedException**.

```
public class Task extends Thread {

    private int id;

    public Task(int id) {
        this.id = id;
    }

    public void run() {
        iterationCounter = 0;
        while (!solved) {
            executeAlgorithmStep();

            try {
                //Resincronizarea thread-urilor pentru urmatorul pas al algoritmului.
                Main.barrier.await();
                ++iterationCounter;
            } catch (BrokenBarrierException | InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

}
```

```
public class Main {

    public static CyclicBarrier barrier;

    public static void main(String[] args) {
        int NUMBER_OF_THREADS = 4;
        barrier = new CyclicBarrier(NUMBER_OF_THREADS);
        Task[] t = new Task[NUMBER_OF_THREADS];

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            t[i] = new Task(id);
            t[i].start();
        }

        for (int i = 0; i < NUMBER_OF_THREADS; ++i) {
            try {
                t[i].join();
```

```
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

CheatSheet CyclicBarrier

Volatile

Cuvântul rezervat **volatile** asociat unei variabile specifică faptul că acea variabilă nu va intra în procesul de cacheing (nici la nivel de regiștrii, nici la nivel de cache L1, L2, L3) și fiecare scriere și citire asociată acesteia se va realiza direct lucrând cu memoria RAM. Acest lucru este util în prevenția citirii unei date vechi din cache-ul sau registrul asociat core-ului pe care rulează un thread în momentul în care variabila a căpătat o valoare nouă pe alt thread ce lucrează pe alt core.

```
public static volatile int counter = 0;
```

Este important de menționat faptul că variabilele volatile cresc timpul de execuție al programului, deoarece fiecare modificare adusă lor trebuie propagată și celorlalte thread-uri, în timp ce variabilele normale pot fi cache-uite la un moment dat chiar până la nivelul regiștrilor procesorului (în cazul unui contor de buclă).

Variabile atomice

În cadrul laboratorului 2 am observat cum operația de incrementare (exemplul cu instrucțiunea a+=2) nu este o operație atomică (operație ce nu poate fi divizată atunci când este executată de către un thread). Java oferă suport pentru o serie de tipuri de date (tipuri atomice) ce au asociate operații atomice [1] [https://docs.oracle.com/javase/8/docs/api/?java/util/concurrent/atomic/package-summary.html], [2] [https://docs.oracle.com/javase/tutorial/essential/concurrency/atomicvars.html]. Acestea sunt utile pentru a fi folosite pe post de contori sau acumulatori fără a mai folosi mecanisme de sincronizate.

Exerciții

1. Creați un program care să lanseze un număr de thread-uri egal cu numărul de core-uri de care dispune calculatorul vostru. Fiecare thread trebuie să afișeze la consolă un text de tipul ”Hello from thread #id”. **[2P]**

2. Rezolvați bug-urile prezente în arhiva de laborator. Folosiți-vă de **hint-uri** din surse. **[3P]**

3. Paralelizați dublarea elementelor unui vector. **[2P]**

4. Paralelizați algoritmul Floyd-Warshall. **[3P]**

Laboratorul 5 - Probleme de sincronizare

wait, notify & notifyAll

Sa presupunem ca avem 2 thread-uri: thread-ul 0 care calculeaza o valoare si o plaseaza in variabila `a` si thread-ul 1 care atunci cand variabila `a` se actualizeaza, o afiseaza in interfata grafica a unui program. O posibila implementare pentru functia thread-ului 0, in pseudocod, ar fi:

```
function_thread_0:
  noua_valoare = calculeaza_valoare_a()
  mutex_a.lock()
      a = noua_valoare
  mutex_a.unlock()
```

O posibila implementare pentru functia thread-ului 1, in pseudocod, ar fi:

```
function_thread_1:
  while(a nu s-a actualizat):
      mutex_a.lock()
      if (a s-a actualizat):
          afiseaza(a)
          a = -1 //  Resetam valoarea lui a.
      mutex_a.unlock()
```

Observam o problema cu aceasta abordare: thread-ul 1 va cicla in bucla while de mai multe ori, chiar daca thread-ul 0 nu-l actualizeaza pe `a`, ocupand inutil timp pe procesor. Aceasta abordare poarta numele de **busy waiting**. Desi **busy waiting** nu este intotdeauna o idee rea, in cazul de fata, prespunuem ca pentru a calcula noua valoarea a lui a, thread-ului 0 ii ia un timp indelungat. Astfel, exista o solutie mai buna oferita de primitevele Java **wait()**, **notify()** și **notifyAll()**.

wait() - forțează thread-ul curent să intre în așteptare până când alt thread apelează **notify()** sau **notifyAll()** pe același obiect. Pentru ca aceasta să se întâmple, thread-ul curent trebuie să dețină monitorul obiectului respectiv. Deținerea monitorului se poate îmtâmpla în următoarele situații:

- s-a executat o metodă synchronized pe obiectul respectiv
- s-a executat un bloc synchronized pe obiectul respectiv
- s-a executat o metodă synchronized statică pe clasa din care face parte obiectul respectiv

Evitați folosirea **wait()** asupra obiectelor cu vizibilitate globală (statice) sau asupra String-urilor cu valoare constanta (de ex: `String myMonitorObject = "";`), deoarece JVM optimizează intern accesul la astfel de variabile, având doar o singură instanță în cadrul programului.

notify() - alege aleatoriu un thread care așteaptă (a apelat **wait()**) monitorul obiectului respectiv și trece-l din starea de *waiting* în starea de *running*

notifyAll() - trece toate thread-urile care așteaptă (au apelat **wait()**) monitorul obiectului respectiv și trece-le din starea de *waiting* în starea de *running*

CheatSheet Wait-Notify

Semafoare

Un semafor poate fi văzut ca un lock ce permite mai multor thread-uri să coexiste într-o anumită regiunie critică la un moment dat. Semaforul folosește un contor care determină câte thread-uri mai pot intra. Odată ajuns la semafor, un thread este lăsat să intre doar dacă numărul de thread-uri aflate în zona critică este mai mic decât numărul maxim de thread-uri setat la crearea semaforului.

Pe un semafor se pot realiza următoarele operații:

- **acquire** - se încearcă trecerea de semafor; dacă numărul de thread-uri aflate în zona critică este mai mic decât numărul maxim de thread-uri acceptate, thread-ul poate intra
- **release** - un thread apeleză metoda pentru a anunța faptul că și-a terminat treaba în zona critică, permițând astfel incrementarea numărului de thread-uri car pot intra în zona critică

```
class Something {

    static int resource = 0;
}

public class MySemaphore extends Thread {

    private int id;
    private Semaphore sem;

    public MySemaphore(int id, Semaphore sem) {
        this.id = id;
        this.sem = sem;
    }

    public static void main(String args[]) {
        Semaphore sem = new Semaphore(1);
```

```
        MySemaphore mt1 = new MySemaphore(0, sem);
        MySemaphore mt2 = new MySemaphore(1, sem);

        mt1.start();
        mt2.start();

        try {
            mt1.join();
            mt2.join();
        } catch (InterruptedException ex) {
            Logger.getLogger(MySemaphore.class.getName()).log(Level.SEVERE, null, ex);
        }

        System.out.println("count: " + Something.resource);
    }

    @Override
    public void run() {
        switch (this.id) {
            case 0:
                System.out.println("Starting thread " + id);
                try {
                    System.out.println("Thread " + id + " is waiting for a permit.");
                    sem.acquire();
                    System.out.println("Thread " + id + " gets a permit.");

                    for (int i = 0; i < 5; i++) {
                        Something.resource++;
                        System.out.println("Thread " + id + ": " + Something.resource);
                        Thread.sleep(10);
                    }
                } catch (InterruptedException exc) {
                    System.out.println(exc);
                }

                System.out.println("Thread " + id + " releases the permit.");
                sem.release();
                break;
            case 1:
                System.out.println("Starting thread " + id);
                try {
                    System.out.println("Thread " + id + " is waiting for a permit.");
                    sem.acquire();
                    System.out.println("Thread " + id + " gets a permit.");

                    for (int i = 0; i < 5; i++) {
                        Something.resource--;
                        System.out.println("Thread " + id + ": " + Something.resource);
                        Thread.sleep(10);
                    }
                } catch (InterruptedException exc) {
                    System.out.println(exc);
                }

                // Release the permit.
                System.out.println("Thread " + id + " releases the permit.");
                sem.release();
```

```
                break;
            }
        }

        public int getThreadId() {
            return this.id;
        }

    }
```

În codul de mai sus se creează un semafor ce acceptă un singur thread în zona critică (observați forma constructorului Semaphore). Practic, acest semafor este similar cu un lock. Semaforul se creează in main si se trimite ca parametru în constructorul thread-urilor. În cadrul metodei run, thread-urile încearcă să apeleze **acquire**, însă doar unul dintre ele va reuși să intre în zona de după semafor. Ulterior, după ce se realizează operațiile din zona critică, thread-ul respectiv anunță faptul că a ieșit prin intermediul unui apel de **release**. Observați faptul că, în funcție de thread-ul care reușește să intre în zona respectivă, contorul poate scădea până la -5, urmând să revină la 0 apoi, sau să fie crescut până la 5, urmând să ajungă la 0 apoi.

CheatSheet Semafoare

Probleme clasice de sincronizare

Producător - Consumator

Problema se referă la două thread-uri: producător și consumator. Producătorul inserează date într-un buffer, iar consumatorul extrage date din acel buffer. Buffer-ul are o dimensiune prestabilită, astfel că:

- producătorul nu poate insera date dacă buffer-ul este plin
- consumatorul nu poate extrage date dacă buffer-ul este gol
- producătorul și consumatorul nu pot acționa simultan asupra buffer-ului

O implementare corectă a problemei presupune asigurarea faptului că nu vor exista situații de deadlock, adică situații în care cele două thread-uri așteaptă unul după celălalt, neexitând posibilitatea de a se debloca.

CheatSheet Producator-Consumator

Problema filozofilor

Problema se referă la mai mulți filozofi (thread-uri) așezați la o masă circulară. Pe masă se află 5 farfurii și 5 tacâmuri, astfel încât fiecare filozof are un tacâm în stânga și unul în dreapta lui. În timp ce stau la masă, filozofii pot face două acțiuni: mănâncă sau se gândesc. Pentru a mânca, un filozof are nevoie de două tacâmuri (pe care le poate folosi doar dacă nu sunt luate de către vecinii săi). Rezolvarea trebuie să aibă în vedere dezvoltarea unui algoritm prin care să nu se ajungă la un deadlock (situația în care fiecare filozof ține câte un tacâm în mână și așteaptă ca vecinul să elibereze celălalt tacâm de care are nevoie).

CheatSheet Problema filozofilor

Căutarea binară paralelă

Algoritmul de căutare binară paralelă se bazează pe împărţirea secvenţei în care se face căutarea în N+1 subsecvenţe de aceeaşi lungime. La fiecare pas, fiecare din cele N procesoare inspectează o valoare s aflată în poziţia dintre două secvenţe adiacente. Dacă `x < s`, atunci toate elementele aflate la dreapta lui s sunt ignorate în pasul următor. Similar, dacă `x > s`, sunt ignorate elementele mai mici ca s. Următorul subinterval de căutare se obţine făcand intersecţia subintervalelor păstrate de diferitele procesoare. Deoarece fiecare etapă se aplică unei secvenţe de dimensiune `1/(N+1)` din secvenţa precedentă, sunt necesare `O(logN+1(n+1))` etape. Pentru a determina subintervalul care rămane în fiecare etapă, fiecare proces *Pi*, `1 ⇐ i ⇐ N` utilizează o variabilă c[i] care ia una din valorile stg sau dr, după cum trebuie reţinută partea din stanga, respectiv cea din dreapta. În plus, se folosesc `c[0] = dr` şi `c[N+1] = stg`. Iniţial, limitele subsecvenţei de căutare sunt `q = 1` şi `r = n`. De asemenea, g are iniţial valoarea numărului maxim de etape: `g = sup(log(n+1)/log(N+1))` şi scade cu o unitate la fiecare iteraţie. Algoritmul calculează valorile c[i] şi găseşte i pentru care c[i-1] <> c[i]. Pe baza lui se determină următorul subinterval [q, r] de căutare:

```
q = q + (i-1) * (N+1) * g - 1 + 1
r = q + i * (N+1) * g - 1 - 1
```

Un singur procesor calculează noile valori pentru q şi r, celelalte având acces la aceste valori în timp constant.

CheatSheet Cautare binara paralela

Exerciții

Schelet laborator 5

1. Pornind de la scheletul **oneProducerOneConsumer** furnizat, implementați algoritmul Producer-Consumer pentru un buffer de dimensiune 1. **[2P]**
2. Modificați algoritmul Producer-Consumer astfel încât să accepte mai mulți producători și mai mulți consumatori. De asemenea, modificați buffer-ul astfel încât să fie de dimensiune > 1. Porniți de la scheletul **multipleProducersMultipleConsumersNBuffer**. **[3P]**
3. Rezolvați problema din algoritmul filozofilor (pachetul **philosophersProblem**) și explicați-o. **[2P]**
4. Implementați căutarea binară paralelă. **[3P]**

Exercițiile din cadrul acestui laborator trebuie rezolvate folosind mecanisme de sincronizare precum primitivele wait/notify/notifyAll sau semafoare. Nu se accepta soluții ce folosesc obiecte concurente.

Laboratorul 6 - Structuri și operații atomice în Java

Responsabili: Radu Ciobanu, Carina Deaconu, Florin Mihalache

Semaphore (continuare)

Laboratorul trecut, am văzut cum semaforul reprezintă o generalizare a unui mutex. Un semafor iniţializat cu 1 poate fi folosit drept lock, pentru că doar un thread are acces la zona critică la un moment de timp. Totuși, un semafor poate avea întrebuinţări mult mai complexe, deoarece poate lua diverse valori, atât negative, cât şi pozitive.

Când un semafor este iniţializat cu valoarea pozitivă *x*, vă puteţi gândi că *x* thread-uri au voie să intre în secţiunea critică. Pe măsură ce un thread face **acquire()**, *x* este decrementat. Când se ajunge la 0, alte posibile thread-uri care vor să acceseze regiunea critică vor trebui să aştepte până când valoarea semaforului creşte la o valoare pozitivă (adică până când iese câte un thread din cele aflate în regiunea critică).

Complementar, când un semafor este iniţializat cu o valoare negativă cum ar fi -1, se aşteaptă ca cel puţin două thread-uri să facă întâi **release()** (pentru ca valoarea semaforului să crească de la -1 la 1), înainte ca o regiune critică să poată fi accesată (adică un alt thread să poată face **acquire()**). Vă puteţi imagina că un al treilea thread aşteaptă „la semafor” ca alte două thread-uri să îi dea un semnal prin apelul **release()** când şi-au „terminat treaba”. Puteți observa un exemplu în pseudocodul de mai jos.

```
Semaphore sem = new Semaphore(-1);
```

T0	T1	T2

`// aşteaptă la semafor`
`// după celelalte 2 thread-uri`
`sem.acquire();`
 	
`// sem = 1, deci poate trece`
`System.out.println(`
`"Am trecut de semafor!");`
	`do_work1();`
	`sem.release();`
	`// sem = -1 + 1 = 0`
		`do_work2();`
		`sem.release();`
		`// sem = 0 + 1 = 1`

AtomicInteger

Clasa AtomicInteger [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/atomic/AtomicInteger.html] poate fi gândită ca o implementare atomică a unui Integer, pentru că reţine o valoare de tip întreg și oferă operații atomice pe ea. Poate fi iniţializat cu o valoare sau nu, valoarea default fiind 0.

Metodele cele mai folosite sunt **get()** (întoarce valoarea de tip întreg), **set(x)** (îi setează valoarea la *x*), **compareAndSet(x, y)** (compară întregul cu *x*, iar, dacă cele două valori sunt egale, setează valoarea la *y*), **addAndGet(delta)** / **getAndAdd(delta)** (incrementează valoarea curentă cu *delta* și returnează valoarea nouă, respectiv pe cea veche). Un exemplu de utilizare a acestor metode poate fi observat mai jos.

```
AtomicInteger nr = new AtomicInteger(4);
System.out.println(nr.get()); // 4

if (nr.compareAndSet(4, 7))
    System.out.println(nr.get()); // nr devine 7

System.out.println(nr.addAndGet(2)); // se aduna 2, deci devine 9

nr.set(1);
System.out.println(nr.get()); // 1
```

Folosirea AtomicInteger este mai avantajoasă ca performanță (mai precis din punctul de vedere al timpilor de execuție) decât folosirea de locks (blocuri synchronized), deoarece la locks avem overhead creat de acquire() (lock()) și de release() (unlock()), așa cum poate fi observat mai jos.

```
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static final int size = 1000000;
    public static final int noThreads = 8;
    public static int[] arr = new int[size];
    public static final Object lock = new Object();

    public static void main(String[] args) {
        for (int i = 0; i < size; i++) {
            arr[i] = i;
        }

        Thread[] properThreads = new Thread[noThreads];
        Thread[] atomicThreads = new Thread[noThreads];

        long startTime = System.nanoTime();
        for (int i = 0; i < properThreads.length; i++) {
            properThreads[i] = new ProperThread(i);
            properThreads[i].start();
        }

        for (Thread properThread : properThreads) {
            try {
                properThread.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        long stopTime = System.nanoTime();
        System.out.println("With locks = " + (stopTime - startTime));

        startTime = System.nanoTime();
        for (int i = 0; i < atomicThreads.length; i++) {
            atomicThreads[i] = new AtomicThread(i);
            atomicThreads[i].start();
        }

        for (Thread atomicThread : atomicThreads) {
            try {
                atomicThread.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        stopTime = System.nanoTime();
        System.out.println("With AtomicInteger = " + (stopTime - startTime));

        System.out.println("Atomic sum = " + AtomicThread.sum.get());
        System.out.println("Locking sum = " + ProperThread.sum);
    }
}
```

```
class AtomicThread extends Thread {
    public static AtomicInteger sum = new AtomicInteger(0);
    private final int id;

    public AtomicThread(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        int start = id * (int) Math.ceil((double) Main.size / Main.noThreads);
        int end = Math.min(Main.size, (id + 1) * (int) Math.ceil((double) Main.size / Main.noThreads));
        for (int i = start; i < end; i++) {
            sum.getAndAdd(Main.arr[i]);
        }
    }
}

class ProperThread extends Thread {
    public static int sum = 0;
    private final int id;

    public ProperThread(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        int start = id * (int) Math.ceil((double) Main.size / Main.noThreads);
        int end = Math.min(Main.size, (id + 1) * (int) Math.ceil((double) Main.size / Main.noThreads));
        for (int i = start; i < end; i++) {
            synchronized (Main.lock) {
                sum += Main.arr[i];
            }
        }
    }
}
```

Java are astfel de implementări atomice precum toate tipurile primitive și nu numai (AtomicBoolean, AtomicLong, AtomicIntegerArray, etc.). Mai multe informații puteți găsi în documentația Java [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/atomic/package-summary.html].

ConcurrentHashMap

ConcurrentHashMap [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html] este o clasă care are aceleaşi funcţionalităţi ca un HashTable (nu HashMap, pentru că nu permite folosi *null* drept cheie sau valoare).

Dacă am folosi un HashMap clasic şi am adăuga un lock pentru accesul la structură, nu ar fi foarte eficient, pentru că doar un singur thread ar putea să o acceseze la un moment de timp, lucru care nu este necesar dacă, de exemplu, ele vor să modifice elemente diferite. Dacă, în schimb, am folosi câte un lock pentru fiecare element, ne-ar trebui destul de multe şi ne-ar îngreuna lucrul.

Într-un ConcurrentHashMap, operaţiile de tip **get** şi **put** blochează accesul la un anumit element (au mutexuri în implementarea lor), dar **NU** blochează accesul la întreaga structură de date. Cu alte cuvinte, două sau mai multe thread-uri o pot accesa în acelaşi timp, şi operaţiile de tip *get* se pot suprapune cu cele de tip update (**put**), dar implementarea se asigură că **get**-urile se execută la final, după ce s-au executat operațiile de update.

Metodele cele mai folosite sunt:

- **get(key)** - returnează valoarea asociată cu o cheie
- **contains(val)** - verifică dacă o valoare există în map
- **containsKey(key)** - verifică dacă o cheie există în map
- **put(key, val)** - adaugă o valoare la cheia specificată
- **putIfAbsent(key, val)** - adaugă o asociere doar dacă nu există deja o valoare pentru cheia respectivă; este întoarsă valoarea veche, dacă există, altfel e întors *null*
- **remove(key)** - şterge o asociere cheie-valoare
- **replace(key, val)** - modifică valoarea asociată cu o cheie.

Puteți observa mai jos un exemplu de utilizare.

```
ConcurrentHashMap<Integer, String> map = new ConcurrentHashMap<Integer, String>();

map.put(0, "Zero");
map.put(1, "One");

System.out.println(map.get(1)); // One

String val = map.putIfAbsent(1, "Unu"); // nu se modifică nimic, val va fi "One"

val = map.putIfAbsent(2, "Two"); // se adaugă maparea 2-"Two", val va fi null
```

BlockingQueue

Interfaţa BlockingQueue [https://docs.oracle.com/en/java/javase/15/docs/api/java.base/java/util/concurrent/BlockingQueue.html] extinde interfaţa Queue, clasele care o implementează fiind potrivite pentru programe paralele, în care pot exista probleme de concurenţă. Dintre acestea, cele mai relevante sunt ArrayBlockingQueue, LinkedBlockingQueue şi PriorityBlockingQueue. Toate simulează o structură de tip FIFO (coadă), cu operaţii thread-safe.

Diferenţa este la implementarea din spate: ArrayBlockingQueue e bazată pe un array, LinkedBlockingQueue pe o listă înlănţuită, iar pentru PriorityBlockingQueue poate fi specificat un comparator la iniţializare, păstrând o ordine în care să fie elementele. Niciuna dintre clase nu acceptă elemente de tip *null* şi li se poate specifica o capacitate maximă la iniţializare (pentru ArrayBlockingQueue este obligatoriu acest lucru).

Operaţiile sunt pe principiul FIFO (se inserează la final şi se extrage de la începutul cozii), așa cum se poate observa în tabelul de mai jos.

Operație	Aruncă excepție	Valoare specială	Se blochează	Dă timeout
Inserare	add(e)	offer(e)	put(e)	offer(e, time, unit)
Ștergere	remove()	poll()	take()	poll(time, unit)
Examinare	element()	peek()	-	-

Deşi sunt mai multe metode pentru aceeaşi operaţie asemănătoare, comportamentul lor este diferit. Cea mai importantă diferenţă o constituie proprietatea de a fi sau nu blocantă. De exemplu, **add(obj)** adaugă un element doar dacă este loc în coadă (adică dacă nu s-a atins capacitatea maximă) şi returnează *true* dacă operaţia a reuşit, sau IllegalStateException în caz contrar. În schimb, **put(obj)** se blochează dacă nu este spaţiu suficient în coadă şi aşteaptă până poate pune elementul cu succes.

Collections.synchronized

Dacă dorim să lucrăm cu colecții clasice, precum ArrayList, și să avem operațiile sincronizate, putem să le instanțiem folosind:

- Collections.synchronizedList (pentru liste):

```
List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());
```

- Collections.synchronizedMap (pentru dicționare):

```
Map<Integer, String> syncMap = Collections.synchronizedMap(new HashMap<>());
```

Arbori concurenți

Atunci când lucrăm cu structuri mai complexe, trebuie să avem grijă la sincronizarea accesului concurent la ele, dar și la eficiența finală a implementării. Ca să vedem un exemplu concret, vom presupune că avem un arbore binar. Când se dorește inserarea unui element într-un astfel de arbore, se verifică mai întâi dacă nodul unde inserăm are copil în partea stângă. Dacă are, atunci nodul copil va fi inserat în stânga, altfel va fi inserat în partea dreaptă:

```
if (node.left == null)
    node.left = child;
else
    node.right = child;
```

Când mai multe thread-uri acționează asupra unui arbore binar, pot apărea probleme de concurență la inserare. Un astfel de scenariu ar putea avea loc atunci când două thread-uri încearcă să insereze simultan câte o valoare diferită în același nod din arbore:

T0	**T1**
Verifică dacă există copil în stânga	Verifică dacă există copil în stânga
Condiția este adevărată, trece în prima ramură a if-ului	Condiția este adevărată, trece în prima ramură a if-ului
Inserează nodul în stânga	
	Inserează nodul în stânga → se suprascrie valoarea scrisă de **T0**

În acest caz, corect ar fi ca unul din thread-uri să insereze în stânga și celălalt să insereze în dreapta. O soluție este să folosim un lock pe operația de inserare unui nod. Dacă folosim un lock global (pentru tot arborele), două thread-uri nu pot insera noduri copii în același timp pentru două noduri părinte diferite. Din acest motiv, mai eficient ar fi să existe câte un lock pentru fiecare nod.

Exerciții

1. Pornind de la arhiva de laborator, rezolvați problema de sincronizare din pachetul *multipleProducersMultipleConsumers*, folosind ArrayBlockingQueue. **(2p)**
2. Rezolvați problema de sincronizare din pachetul *synchronizationProblem*, folosind un AtomicInteger. **(2p)**
3. Rezolvați problema de sincronizare din pachetul *bugConcurrentHashMap*, folosind metode din clasa ConcurrentHashMap. **(2p)**

4. Paralelizați programul din pachetul *synchronizedSortedList*, unde:
    a. trei thread-uri citesc numere din trei fișiere (câte un thread citește câte un fișier) și adaugă numerele din fișiere într-o listă partajată
    b. al patrulea thread sortează lista cu numerele
    c. lista poate fi sortată **doar** după ce citirea a fost efectuată de către toate cele trei thread-uri care citesc din fișier
    d. sincronizarea între thread-uri trebuie realizată folosind un semafor. **(2p)**
5. Paralelizați programul din pachetul *parallelTree*, unde:
    a. două thread-uri construiesc un arbore binar în paralel, citind ID-urile și părintele nodurilor noi din câte un fișier, și apoi executând operația de inserare
    b. al treilea verifică dacă arborele a fost construit corect
    c. verificarea arborelui nu se poate face decât după finalizarea alcătuirii sale
    d. sincronizarea între thread-uri trebuie realizată folosind o barieră
    e. la inserția în arbore, sincronizarea nu trebuie făcută global (pe tot arborele), ci pentru fiecare nod din arbore. **(2p)**

Laboratorul 7 - Modelul Replicated Workers

Responsabili: Radu Ciobanu, Carina Deaconu, Florin Mihalache

În programarea paralelă, modelul **Replicated Workers** (sau **Thread Pool**) este un design pattern folosit pentru obținerea de concurență în execuția unui program. În acest model, există două componente principale: un **pool** de sarcini de executat (task-uri), reprezentat în general ca o coadă, și un grup de **workeri** (care sunt în general thread-uri). Fiecare worker din modelul replicated workers are următorul comportament:

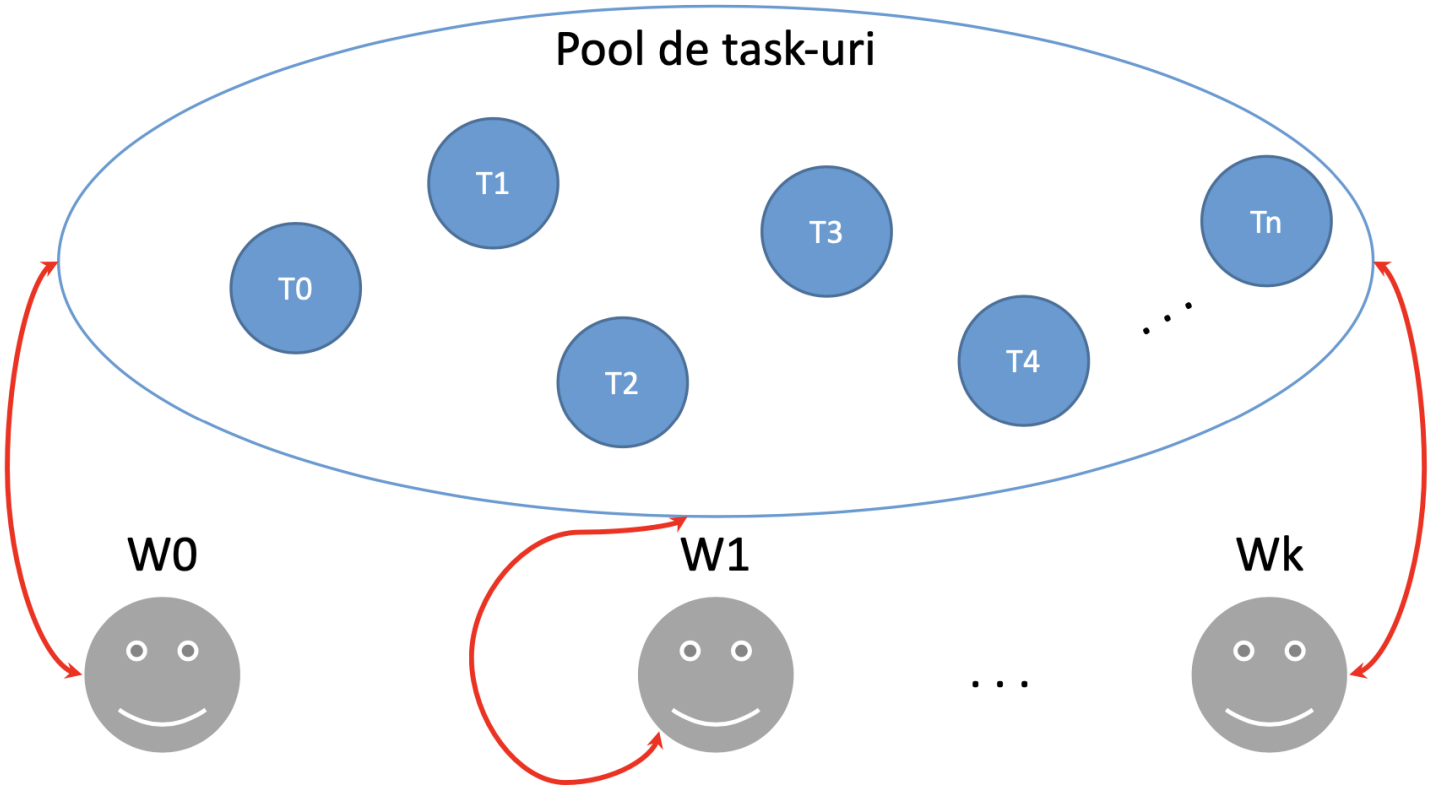
```
while (1)
    ia un task din pool
    execută task-ul
    adaugă zero sau mai multe task-uri rezultate în pool
```

Așa cum se poate observa în pseudocodul de mai sus, un worker ia un task din pool și îl execută. Ca urmare a execuției unui task, este posibil să se genereze task-uri adiționale, pe care worker-ul le adaugă în pool, după care repetă pașii precedenți.

Modelul Replicated Workers este util atunci când:

- nu se cunoaște în avans dimensiunea problemei, adică numărul de pași care se vor executa
- există multe task-uri de dimensiuni mici care trebuie executate și este mai eficient să se refolosească aceleași thread-uri decât să se creeze unele noi la fiecare pas
- există posibilitatea ca thread-urile să fie dezechilibrate (de exemplu, unele core-uri sunt mai ocupate, sau chiar diferite între ele, și o împărțire egală a sarcinilor de calcul ar duce la întârzieri în program).

O reprezentare grafică a modelului replicated workers poate fi observată în figura de mai jos.



ExecutorService

ExecutorService [https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/ExecutorService.html] este o interfață în Java ce permite executarea de task-uri asincrone, în background, în mod concurent, pe baza modelului Replicated Workers. De exemplu, putem avea nevoie să trimitem un număr de cereri, dar ar fi ineficient să le trimitem pe rând, secvențial, așteptând după fiecare să se termine. Soluția ar fi să lucrăm asincron, adică să trimitem o cerere, să nu așteptăm „după ea” (deci să se trimită în background) și să folosim thread-uri pentru a împărți numărul de cereri și a trimite mai multe deodată (concurent). Acesta ar fi cazul când nu știm în avans dimensiunea problemei pe care o rezolvăm, și fie avem nevoie de toate soluțiile problemei (pentru că trebuie să trimitem toate cererile), fie nu vrem să găsim toate soluțiile, ci minim una.

În continuare, se prezintă un exemplu de program care folosește ExecutorService pentru a afișa toate fișierele dintr-un director (inclusiv din fiecare subdirector în parte). Pentru că nu știm în avans câte fișiere și directoare va trebui să analizăm, putem spune că nu cunoaștem dimensiunea problemei.

```
import java.io.File;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Example1 {
    public static void main(String[] args) {
        AtomicInteger inQueue = new AtomicInteger(0);
        ExecutorService tpe = Executors.newFixedThreadPool(4);
```

```
        inQueue.incrementAndGet();
        tpe.submit(new MyRunnable("files", tpe, inQueue));
    }
}

class MyRunnable implements Runnable {
    String path;
    ExecutorService tpe;
    AtomicInteger inQueue;

    public MyRunnable(String path, ExecutorService tpe, AtomicInteger inQueue) {
        this.path = path;
        this.tpe = tpe;
        this.inQueue = inQueue;
    }

    @Override
    public void run() {
        File file = new File(path);
        if (file.isFile()) {
            System.out.println(file.getPath());
        } else if (file.isDirectory()) {
            File[] files = file.listFiles();

            if (files != null) {
                for (File f : files) {
                    inQueue.incrementAndGet();
                    tpe.submit(new MyRunnable(f.getPath(), tpe, inQueue));
                }
            }
        }

        int left = inQueue.decrementAndGet();
        if (left == 0) {
            tpe.shutdown();
        }
    }
}
```

Așa cum se poate observa în codul de mai sus, instanțierea unui ExecutorService cu 4 workeri se realizează în thread-ul principal. Prin metoda *submit()*, se introduce un task nou în pool, corespunzător directorului părinte. Metoda *submit()* poate primi un obiect ce implementează interfața Runnable și conține metoda *run()* sau unul care implementează interfața Callable și conține metoda *call()*. Diferența dintre ele este că *run()* returnează void, iar *call()* returnează un obiect și poate arunca și o excepție. Cu alte cuvinte, Callable este potrivit când ne interesează rezultatul final și eventualele erori la executarea task-ului. În cadrul laboratorului, vom lucra cu interfața Runnable.

În exemplul de mai sus, clasa noastră ce reprezintă un task se numește MyRunnable. În metoda run, se verifică dacă avem de-a face cu un fișier sau cu un director. Dacă lucrăm pe un fișier, doar se printează numele acestuia. Dacă lucrăm cu un director, înseamnă că trebuie create task-uri noi, pentru fiecare fișier sau subdirector în parte conținut de directorul părinte. Astfel, se creează câte o instanță de MyRunnable pentru fiecare din ele și se adaugă în pool-ul de ExecutorService.

În cazul exemplului prezentat în acest laborator, ne trebuie toate soluțiile problemei (toate fișierele și subdirectoarele), dar nu știm dinainte dimensiunea problemei, așa că o

metodă prin care ne putem opri este să ținem evidența task-urilor din pool și să finalizăm execuția atunci când acesta se golește. Altfel, programul nostru va rula la infinit. Modul prin care se finalizează execuția unui ExecutorService poate fi observat în codul de mai sus. Mai precis, se folosește metoda *shutdown()* pentru a opri ExecutorService-ul din a primi task-uri noi. Dacă ne-ar fi interesat o singură soluție a problemei noastre (și nu toate, ca în exemplul de mai sus), am fi putut opri execuția în momentul în care ajungeam la acea soluție.

ForkJoinPool

O altă metodă de a implementa modelul Replicated Workers în Java este framework-ul Fork/Join, care folosește o abordare *divide et impera*, ceea ce înseamnă că întâi are locul procesul de împărțire (Fork) recursivă a task-ului inițial în subtask-uri independente mai mici, până când acestea sunt suficient de mici cât să poată fi executate asincron. După aceea, urmează partea de colectare recursivă a rezultatelor (Join) într-un singur rezultat (în cazul unui task care nu returnează un rezultat propriu-zis, pur și simplu se așteaptă ca toate subtask-urile să se termine de executat).

Pentru o execuție paralelă eficientă, framework-ul Fork/Join folosește un pool de thread-uri numit ForkJoinPool [https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/ForkJoinPool.html], care gestionează thread-uri worker de tipul ForkJoinWorkerThread. ForkJoinPool este o implementare de ExecutorService care gestionează thread-uri de tip worker, care pot executa câte un singur task la un moment dat de timp. În implementarea sa, fiecare thread are propria sa coadă de task-uri, dar, atunci când aceasta se golește, worker-ul poate să „fure” task-uri din coada altui worker sau din pool-ul global de task-uri.

Un task care va fi executat folosind framework-ul Fork/Join poate fi definit în două moduri, în funcție de valoarea de retur. Dacă task-ul nu trebuie să returneze nimic, un task se definește prin moștenirea clasei RecursiveAction. Dacă task-ul returnează o valoare de tip *V*, atunci trebuie să se moștenească clasa RecursiveTask<V> pentru definirea unui task. Ambele clase părinte au o metodă numită *compute()* în care se definește logica unui task (echivalentul metodei *run()* de la ExecutorService).

Pentru a adăuga task-uri care trebuie executate de workeri, se poate folosi metoda *invoke()*, care creează un task și îi așteaptă rezultatul, sau combinația de metode *fork()* și *join()*. Pentru al doilea caz, metoda *fork()* trimite un task în pool, dar nu îl marchează spre execuție, acest lucru făcându-se explicit prin intermediul metodei *join()*. Atât *invoke()*, cât și *join()*, returnează o valoare de tipul *V* pentru un task de tip RecursiveTask.

Pentru că, în cazul unui task care nu returnează nimic, se poate folosi RecursiveTask<V>, prezentăm mai jos un exemplu complet de implementare care folosește RecursiveTask, pentru aceeași problemă ca în exemplul de ExecutorService.

```
import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;
```

```
public class Example2 {
    public static void main(String[] args) {
        ForkJoinPool fjp = new ForkJoinPool(4);
        fjp.invoke(new MyTask("files"));
        fjp.shutdown();
    }
}

class MyTask extends RecursiveTask<Void> {
    private final String path;

    public MyTask(String path) {
        this.path = path;
    }

    @Override
    protected Void compute() {
        File file = new File(path);
        if (file.isFile()) {
            System.out.println(file.getPath());
            return null;
        } else if (file.isDirectory()) {
            File[] files = file.listFiles();
            List<MyTask> tasks = new ArrayList<>();
            if (files != null) {
                for (File f : files) {
                    MyTask t = new MyTask(f.getPath());
                    tasks.add(t);
                    t.fork();
                }
            }
            for (MyTask task : tasks) {
                task.join();
            }
        }

        return null;
    }
}
```

În exemplul de mai sus, se pot observa ambele moduri de a crea și submite spre execuție un task nou. În *main()*, se folosește *invoke()*, care se va bloca până când toate task-urile s-au terminat, deci până este sigur să apelăm *shutdown()*. În schimb, în interiorul unui task, se adaugă toate task-urile folosind *fork()*, și apoi se așteaptă finalizarea lor folosind *join()*.

Exerciții

Hint

În acest laborator, veți lucra cu această arhivă, pe care o veți utiliza să paralelizați trei probleme pe baza modelului Replicated Workers, folosind ExecutorService și ForkJoinPool:

- găsirea căilor dintre nodurile unui graf
- colorarea unui graf → găsiți detalii adiționale aici

- problema damelor → găsiți detalii adiționale aici.

1. Paralelizați găsirea căilor între două noduri pe baza scheletului oferit (în pachetul *task1*) folosind ExecutorService. **(1p)**
2. Paralelizați problema colorării unui graf pe baza scheletului oferit (în pachetul *task2*) folosind ExecutorService. **(2p)**
3. Paralelizați problema damelor pe baza scheletului oferit (în pachetul *task3*, cu soluțiile *[(2, 1), (4, 2), (1, 3), (3, 4)]* și *[(3, 1), (1, 2), (4, 3), (2, 4)]*) folosind ExecutorService. **(2p)**
4. Paralelizați găsirea căilor între două noduri pe baza scheletului oferit (în pachetul *task4*) folosind ForkJoinPool. **(1p)**
5. Paralelizați problema colorării unui graf pe baza scheletului oferit (în pachetul *task5*) folosind ForkJoinPool. **(2p)**
6. Paralelizați problema damelor pe baza scheletului oferit (în pachetul *task6*) folosind ForkJoinPool. **(2p)**

Laboratorul 8 - Introducere în programarea distribuită cu MPI

Despre MPI și programarea distribuită

Până acum, ați lucrat cu programarea paralelă, care presupune existența a mai multor fire de execuție (threads), care execută instrucțiuni în mod paralel și concurent, ele având acces la aceeași zonă de memorie, în cadrul unei singure mașini de calcul, care are mai multe procesoare.

Putem extinde conceptul de programare paralelă prin folosirea a mai multor mașini de calcul, care sunt conectate în cadrul unei rețele într-un concept numit **programarea distribuită**.

Spre deosebire de programarea paralelă, în programarea distribuită nu există conceptul de memorie partajată (shared memory). Întrebarea este următoarea: cum poate o mașină să știe ce date are o altă mașină din cadrul rețelei? Soluția este **comunicarea prin mesaje**, realizată de către mașinile (de tip nod) dintr-o rețea. Comunicarea prin mesaje are două funcționalități:

- comunicarea: un nod (transmițător / sender) trimite date prin intermediul unui canal de comunicație către un alt nod (receptor / receiver)
- sincronizarea: un mesaj nu poate fi recepționat înainte ca acesta să fie transmis

MPI (Message Passing Interface) reprezintă un standard pentru comunicarea prin mesaje, elaborat de MPI Forum, și are la bază modelul proceselor comunicante prin mesaje.

Un proces reprezintă un program aflat în execuție și se poate defini ca o unitate de bază care poate executa una sau mai multe sarcini în cadrul unui sistem de operare. Spre deosebire de thread-uri, un proces are propriul său spațiu de adrese (propria zonă de memorie) și acesta poate avea, în cadrul său, mai multe thread-uri în execuție, care partajează resursele procesului.

În cadrul lucrului în C/C++, MPI reprezintă o bibliotecă, care are funcționalitățile implementate într-un header numit **mpi.h**. Pentru compilare, la MPI există un compilator specific:

- mpicc, pentru lucrul în C
- mpic++, pentru lucrul în C++

În ambele limbaje, pentru rularea unui program MPI folosim comanda **mpirun**, împreună cu parametrul **-np**, unde precizăm numărul de procese care rulează în cadrul programului distribuit.

Exemplu:

- compilare:
  - C: `mpicc hello.c -o hello`
  - C++: `mpic++ hello.cpp -o hello`
- rulare: `mpirun -np 4 hello` - rulare cu 4 procese

Dacă încercați să rulați comanda **mpirun** cu un număr de procese mai mare decât numărul de core-uri fizice disponibile pe procesorul vostru, este posibil să primiți o eroare cum ca nu aveți destule sloturi libere. Puteți elimina acea eroare adăugând parametrul **–oversubscribe** atunci când rulați mpirun.

Instalare MPI: Pentru a lucra cu MPI, trebuie să instalați biblioteca pentru MPI pe Linux, folosind următoarea comandă: `sudo apt install openmpi-bin openmpi-common openmpi-doc libopenmpi-dev`

În caz că lucrați cu MPI pe WSL, o să vă apară un warning, care poate fi fixat cu ajutorul hint-urilor de aici [https://github.com/microsoft/WSL/issues/3397]

Implementarea unui program distribuit în MPI

Exemplu de program MPI - Hello World:

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

#define  MASTER 0

int main (int argc, char *argv[]) {
    int numtasks, rank, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Get_processor_name(hostname, &len);
    if (rank == MASTER)
        printf("MASTER: Number of MPI tasks is: %d\n",numtasks);
    else
        printf("WORKER: Rank: %d\n",rank);

    MPI_Finalize();
}
```

Un comunicator (`MPI_Comm`) reprezintă un grup de procese care comunică între ele.
`MPI_COMM_WORLD` reprezintă comunicatorul default, din care fac parte toate procesele.

Funcții:

- `MPI_Init` - se inițializează programul MPI, mai precis se creează contextul în cadrul căruia rulează procesele. Argumentele din linie de comandă sunt pasate către contextul de rulare a proceselor.
- `MPI_Comm_size` - funcție care determină numărul de procese (numtasks) care rulează în cadrul comunicatorului (de regulă MPI_COMM_WORLD)
- `MPI_Comm_rank` - funcție care determină identificatorul (rangul) procesului curent în cadrul comunicatorului.
- `MPI_Get_processor_name` - determină numele procesorului
- `MPI_Finalize` - declanșează terminarea programului MPI

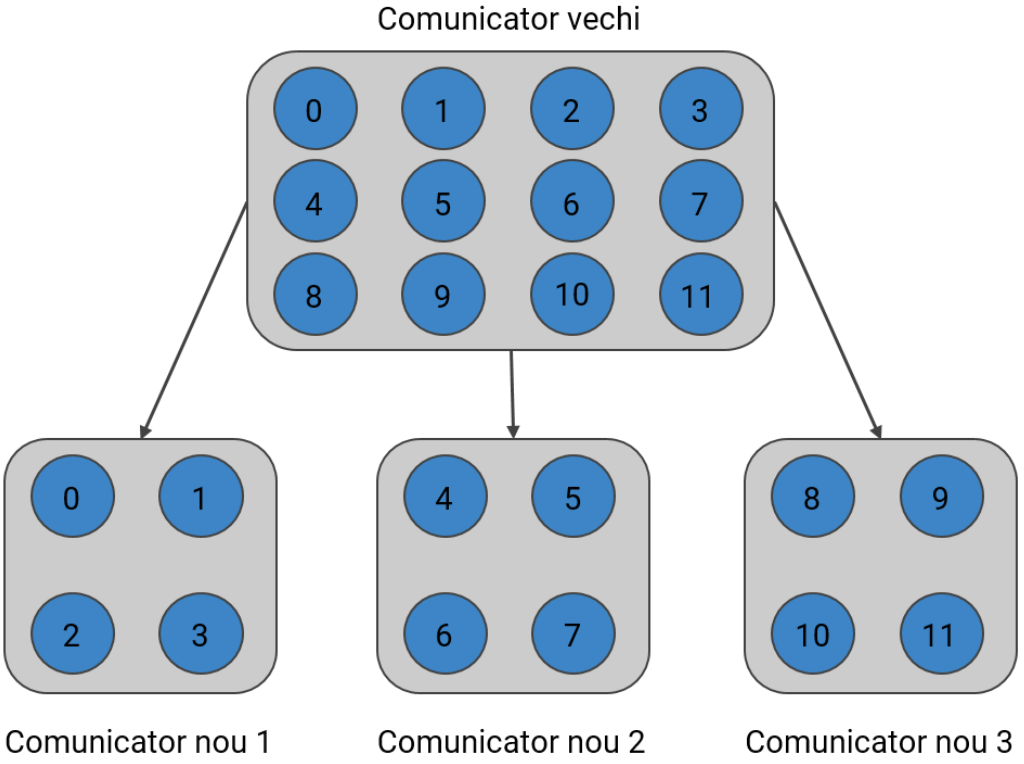
În cadrul schimbului de date între procese, este necesar mereu să precizăm tipul acestora. În MPI, se folosește enum-ul `MPI_Datatype`, care se mapează cu tipurile de date din C/C++, după cum puteți vedea în tabelul de mai jos:

MPI_Datatype	Echivalentul din C/C++
MPI_INT	int
MPI_LONG	long
MPI_CHAR	char
MPI_FLOAT	float
MPI_DOUBLE	double

Putem crea comunicatoare dintr-un alt comunicator folosind funcția MPI_Comm_split, care împarte un comunicator în mai multe comunicatoare mai mici. Semnătura funcției este următoarea: `int MPI_Comm_split(MPI_Comm comm, int color, int key, MPI_Comm * newcomm)`, unde:

- `comm` - comunicatorul este împărțit
- `color` - identificator al noului comunicator, din care face parte un proces (de regulă rang_vechi_proces / dimensiune_comunicator_nou)
- `key` - noul rang al procesului în cadrul noului comunicator (de regulă rang_vechi_proces % dimensiune_comunicator_nou)
- `newcomm` - noul comunicator format

Aveți mai jos o imagine care ilustrează cum funcționează `MPI_Comm_split`:



Funcții de transmisie a datelor în MPI

Convenție: vom marca cu ↓ parametrii de tip input ai unei funcții și cu ↑ parametrii de tip output în cadrul prezentării funcțiilor de mai jos.

MPI_Send

MPI_Send reprezintă funcția prin care un proces trimite date către un alt proces. Semnătura funcției este următoarea:

`int MPI_Send(void* data, int count, MPI_Datatype datatype, int destination, int tag, MPI_Comm communicator)`, unde:

- `data` (↓) - reprezintă datele trimise de la procesul sursă către procesul destinație
- `count` (↓) - dimensiunea datelor transmise
- `datatype` (↓) - tipul datelor transmise
- `destination` (↓) - rangul / identificatorului procesului destinație, către care se trimit datele

- `tag` (↓) - identificator al mesajului
- `communicator` (↓) - comunicatorul în cadrul căruia se face trimiterea datelor între cele două procese

MPI_Send este o funcție blocantă. Mai precis, programul se blochează până când bufferul dat ca prim parametru poate fi refolosit, chiar dacă nu se execută acțiunea de primire a mesajului transmis de procesul curent (MPI_Recv). Dacă apare cazul în care procesul P1 trimite date (MPI_Send) la procesul P2, iar P2 nu are suficient loc în buffer-ul de recepție (buffer-ul nu are suficient loc liber sau este plin) atunci P1 se va bloca.

MPI_Recv

MPI_Recv reprezintă funcția prin care un proces primește date de la un alt proces. Semnătura funcției este următoarea:

`int MPI_Recv(void* data, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm communicator, MPI_Status* status)`, unde:

- `data` (↑) - reprezintă datele primite de la procesul sursă de către procesul destinație
- `count` (↓) - dimensiunea datelor primite
- `datatype` (↓) - tipul datelor primite
- `source` (↓) - rangul / identificatorului procesului sursă, care trimite datele
- `tag` (↓) - identificator al mesajului
- `communicator` (↓) - comunicatorul în cadrul căruia se face trimiterea datelor între cele două procese
- `status` - conține date despre mesajul primit, `MPI_Status` fiind o structură ce conține informații despre mesajul primit (sursa, tag-ul mesajului, dimensiunea mesajului). Dacă nu dorim să ne folosim de datele despre mesajul primit, punem `MPI_STATUS_IGNORE`, prin care se ignoră status-ul mesajului.

În situația în care procesul P apelează funcția de MPI_Recv(), el se va bloca până va primi toate datele asteptate, astfel că dacă nu va primi nimic sau ceea ce primește este insuficient, P va rămâne blocat. Adică MPI_Recv() se termină doar în momentul în care buffer-ul a fost umplut cu datele așteptate.

Structura `MPI_Status` include următoarele câmpuri:

- `int count` - dimensiunea datelor primite
- `int MPI_SOURCE` - identificatorul procesului sursă, care a trimis datele
- `int MPI_TAG` - tag-ul mesajului primit

MPI_Recv este o funcție blocantă, mai precis programul se poate bloca până când se execută acțiunea de trimitere a mesajului către procesul sursă.

Un exemplu de program în care un proces trimite un mesaj către un alt proces:

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
    int  numtasks, rank, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks); // Total number of processes.
```

```
    MPI_Comm_rank(MPI_COMM_WORLD,&rank); // The current process ID / Rank.
    MPI_Get_processor_name(hostname, &len);

    srand(42);
    int random_num = rand();
    printf("Before send: process with rank %d has the number %d.\n", rank,
            random_num);

    if (rank == 0) {
        MPI_Send(&random_num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else {
        MPI_Status status;
        MPI_Recv(&random_num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process with rank %d, received %d with tag %d.\n",
                rank, random_num, status.MPI_TAG);
    }

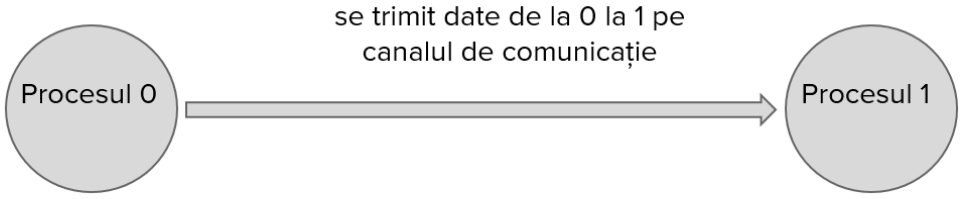
    printf("After send: process with rank %d has the number %d.\n", rank,
            random_num);

    MPI_Finalize();

}
```

Când un proces X trimite un mesaj către un proces Y, tag-ul T al mesajului din MPI_Send, executat de procesul X, trebuie să fie același cu tag-ul mesajului din MPI_Recv, executat de procesul Y, deoarece procesul Y așteaptă un mesaj care are tag-ul T, altfel, dacă sunt tag-uri diferite, programul se va bloca.

O ilustrație a modului cum funcționează împreună funcțiile MPI_Send și MPI_Recv:



Execută MPI_Send (trimite date la procesul 1) Execută MPI_Recv (primește date de la procesul 0)

Mai jos aveți un exemplu în care un proces trimite un întreg array de 100 de elemente către un alt proces:

```
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv[])
{
    int numtasks, rank, len;
    int size = 100;
    char hostname[MPI_MAX_PROCESSOR_NAME];
    int arr[size];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);
    MPI_Get_processor_name(hostname, &len);

    srand(42);
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            arr[i] = i;
        }
```

```
        printf("Process with rank [%d] has the following array:\n", rank);
        for (int i = 0; i < size; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");

        MPI_Send(&arr, size, MPI_INT, 1, 1, MPI_COMM_WORLD);
        printf("Process with rank [%d] sent the array.\n", rank);
    } else {
        MPI_Status status;
        MPI_Recv(&arr, size, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        printf("Process with rank [%d], received array with tag %d.\n",
                rank, status.MPI_TAG);

        printf("Process with rank [%d] has the following array:\n", rank);
        for (int i = 0; i < size; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }

    MPI_Finalize();

}
```

MPI_Bcast

MPI_Bcast reprezintă o funcție prin care un proces trimite un mesaj către toate procesele din comunicator (message broadcast), inclusiv lui însuși.

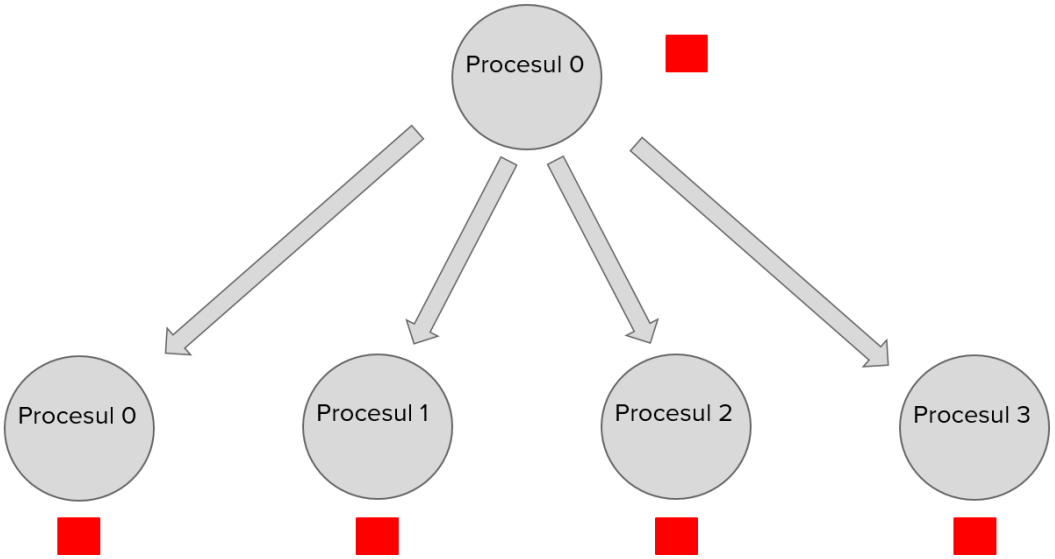
În cadrul implementării MPI_Bcast sunt executate acțiunile de trimitere și de recepționare de mesaje, așadar nu trebuie să apelați MPI_Recv.

Semnătura funcției este următoarea:

`int MPI_Bcast(void* data, int count, MPI_Datatype datatype, int root, MPI_Comm communicator)`, unde:

- `data` (↓ + ↑) - reprezintă datele care sunt transmise către toate procesele. Acest parametru este de tip input pentru procesul cu identificatorul `root` și este de tip output pentru restul proceselor.
- `count` (↓) - dimensiunea datelor trimise
- `datatype` (↓)- tipul datelor trimise
- `root` (↓) - rangul / identificatorului procesului sursă, care trimite datele către toate procesele din comunicator, inclusiv lui însuși
- `tag` (↓) - identificator al mesajului
- `communicator` (↓) - comunicatorul în cadrul căruia se face trimiterea datelor către toate procesele din cadrul acestuia

O ilustrație care arată cum funcționează MPI_Bcast aveți mai jos:



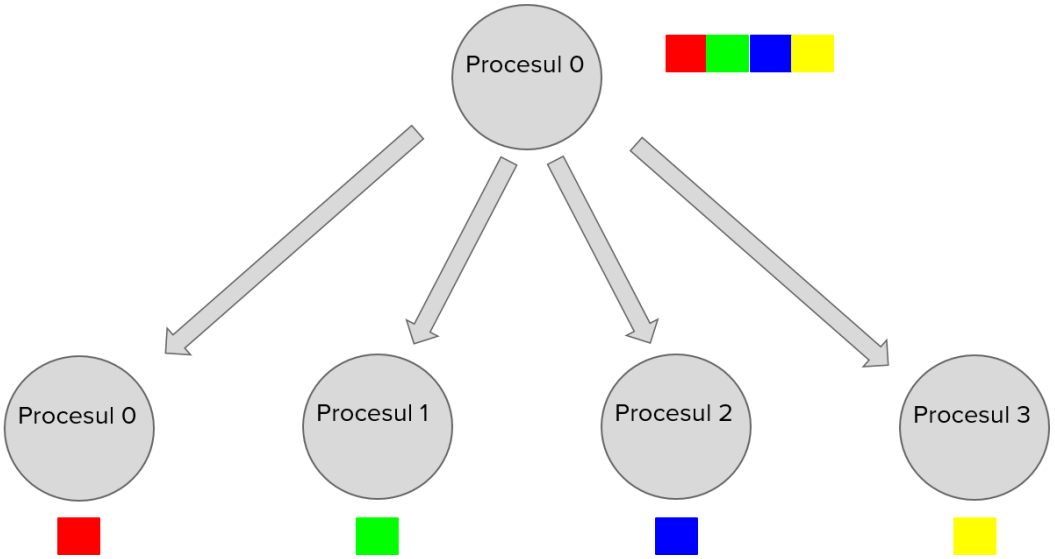
MPI_Scatter

MPI_Scatter este o funcție prin care un proces împarte un array pe bucăți egale ca dimensiuni, unde fiecare bucată revine, în ordine, fiecărui proces, și le trimite tuturor proceselor din comunicator, inclusiv lui însuși.

Semnătura funcției este următoarea: `int MPI_Scatter(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)`, unde:

- `send_data` (↓) - reprezintă datele care sunt împărțite și trimise către procesele din comunicator
- `send_count` (↓) - reprezintă dimensiunea bucății care revine fiecărui proces (de regulă se pune ca fiind dimensiunea_totală / număr_de_procese).
- `send_datatype` (↓) - tipul datelor trimise către procese
- `recv_data` (↑) - reprezintă datele care sunt primite și stocate de către procese
- `recv_count` (↓) - dimensiunea datelor primite (de regulă dimensiunea_totală / număr_de_procese)
- `recv_datatype` (↓) - tipul datelor primite de către procese (de regulă este același cu send_datatype)
- `root` (↓) - identificatorul procesului care împarte datele și care le trimite către procesele din comunicator, inclusiv lui însuși
- `communicator` (↓) - comunicatorul din care fac parte procesele (de regulă `MPI_COMM_WORLD`)

O ilustrație a modului cum funcționează MPI_Scatter:



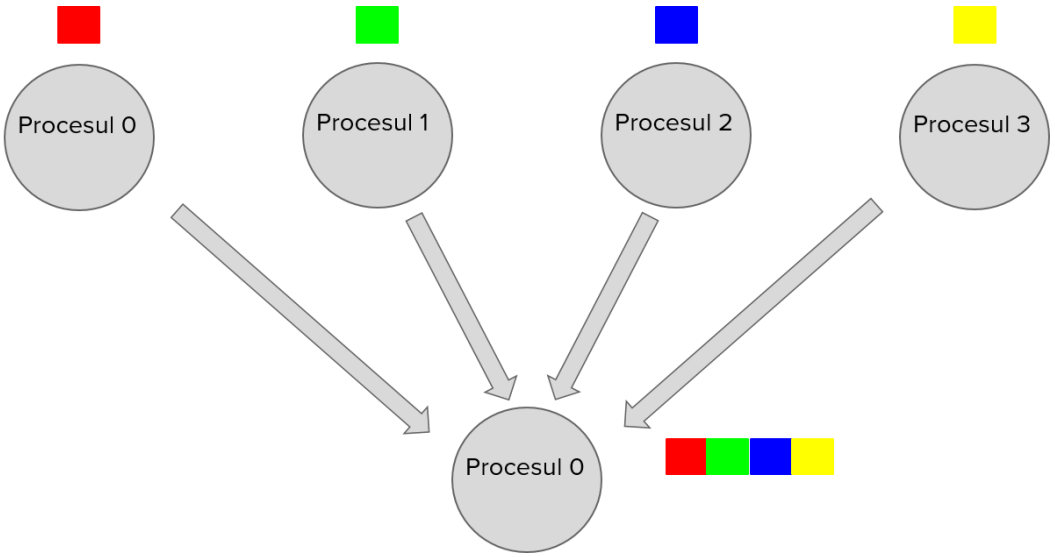
MPI_Gather

MPI_Gather este o funcție care reprezintă inversul lui MPI_Scatter, în sensul că un proces primește elemente de la fiecare proces din comunicator, inclusiv de la el însuși, și le unifică într-o singură colecție.

Semnătura funcției este următoarea: `int MPI_Gather(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)`, unde:

- `send_data` (↓) - reprezintă datele care trimise de fiecare proces către procesul cu id-ul root
- `send_count` (↓) - reprezintă dimensiunea bucății trimisă de fiecare proces (de regulă se pune ca fiind dimensiunea_totală / număr_de_procese).
- `send_datatype` (↓) - tipul datelor trimise de către procese
- `recv_data` (↑) - reprezintă datele care sunt primite și stocate de către procesul root
- `recv_count` (↓) - dimensiunea datelor primite (de regulă dimensiunea_totală / număr_de_procese)
- `recv_datatype` (↓) - tipul datelor primite de către procesul root (de regulă este același cu send_datatype)
- `root` (↓) - identificatorul procesului care primește datele (inclusiv de la el însuși)
- `communicator` (↓) - comunicatorul din care fac parte procesele (de regulă `MPI_COMM_WORLD`)

O ilustrare a modului cum funcționează MPI_Gather:



Mai jos aveți un exemplu de MPI_Scatter folosit împreună cu MPI_Gather:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ROOT 0
#define CHUNK_SIZE 5 // numarul de elemente per proces

int main (int argc, char **argv) {
    int rank, proc, a;

    int* arr;
    int* process_arr;
    int* result_arr;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &proc);

    if (rank == ROOT) {
        arr = malloc (CHUNK_SIZE * proc * sizeof(int));
        for (int i = 0; i < proc * CHUNK_SIZE; ++i) {
            arr[i] = 0;
        }
    }

    process_arr = malloc (CHUNK_SIZE * sizeof(int));
    MPI_Scatter(arr, CHUNK_SIZE, MPI_INT, process_arr, CHUNK_SIZE, MPI_INT, ROOT, MPI_COMM_WORLD);

    for (int i = 0; i < CHUNK_SIZE; i++) {
        printf("Before: rank [%d] - value = %d\n", rank, process_arr[i]);
        process_arr[i] = i;
        printf("After: rank [%d] - value = %d\n", rank, process_arr[i]);
    }

    if (rank == ROOT) {
        result_arr = malloc (CHUNK_SIZE * proc * sizeof(int));
    }

    MPI_Gather(process_arr, CHUNK_SIZE, MPI_INT, result_arr, CHUNK_SIZE, MPI_INT, ROOT, MPI_COMM_WORLD);

    if (rank == ROOT) {
        for (int i = 0; i < CHUNK_SIZE * proc; i++) {
            printf("%d ", result_arr[i]);
        }
        printf("\n");
    }
```

```
    if (rank == ROOT) {
        free(arr);
        free(result_arr);
    }

    free(process_arr);

    MPI_Finalize();
    return 0;
}
```

Algoritmul inel

Algoritmul inel este un algoritm de tip undă, care funcționează pe topologii de sub formă de inel, unde fiecare nod proces are un vecin dedicat (Urm / next). În cadrul topologiilor inel, transmisia datelor se face folosind adresarea prin direcție, unde mai precis un nod proces trimite datele către vecinul său (Urm), astfel în cadrul topologiei formându-se un ciclu Hamiltonian.

În cadrul algoritmului, avem un nod proces inițiator, care trimite un jeton către vecinul său, care este pasat de fiecare proces către vecinul său de-a lungul ciclului, până când jetonul ajunge înapoi până la nodul proces inițiator.

Pseudocod:

```
chan token[1..n] (tok_type tok) //  canalul de comunicatie

// initiator
process P[i] {
    tok_type tok;
    send token[next](tok); // trimite token-ul la urmatorul nod
    receive token[i](tok); // primeste token-ul de la nodul precedent
    decizie;
}

// neinitiator
process P[k, k = 1..n, k != n] {
    tok_type tok;
    receive token[k](tok); // primeste token-ul de la nodul precedent
    send token[next](tok); // trimite token-ul la urmatorul nod
}
```

Pentru a vedea execuția pe pași al acestui algoritm, aveți această prezentare la dispoziție.

Exerciții

- **Exercițiul 1 (2p):** Implementați algoritmul inel folosind MPI, unde procesul cu id-ul 0 trimite un număr aleatoriu către procesul 1 (vecinul său), iar apoi celelalte noduri vor primi numărul de la procesul precedent, îl incrementează cu 2 și îl trimit la următorul proces (de exemplu procesul 2 primește valoarea de la procesul 1, o incrementează și o trimite mai departe procesului 3), totul terminându-se când valoarea ajunge la procesul 0. Pentru fiecare proces trebuie să afișați rangul acestuia și valoarea primită.
- **Exercițiul 2 (2p):** Implementați un program în MPI în care procesul 0 trimite o valoare generată aleatoriu către celelalte procese, folosind `MPI_Bcast`. **Atenție:** Nu folosiți `MPI_Recv`.
- **Exercițiul 3 (2p):** Implementați un program în MPI cu N procese, în care procesul 0 inițializează un array de dimensiune 5 * N cu 0, îl împarte (dimensiunea chunk-ului fiind 5, definită în schelet) și îl trimite tuturor proceselor cu `MPI_Scatter`. Procesele vor aduna fiecare element primit cu rangul fiecăruia, apoi ele vor trimite array-urile pe care au efectuat operația

de adunare cu rangul către procesul 0 folosind `MPI_Gather`. Procesul 0 va afișa ulterior array-ul rezultat din `MPI_Gather`.
- **Exercițiul 4 (2p):** Implementați un program MPI cu 4 procese, unde 3 procese trimit o valoare către al patrulea proces, care va primi valoarea folosind `MPI_ANY_SOURCE` și va afișa valoarea primită și rangul procesului sursă folosind `MPI_SOURCE` din `MPI_Status` (status din `MPI_Recv`).

`MPI_ANY_SOURCE` este folosit în loc de rangul procesului sursă la `MPI_Recv`, indicând astfel că procesul receptor poate primi un mesaj de la oricare proces.

- **Exercițiul 5 (2p):** Implementați un program MPI cu 2 procese, unde procesul 0 trimite 10 valori cu tag-uri diferite către procesul 1, iar procesul 1 primește valorile folosind parametrul `MPI_ANY_TAG` în `MPI_Recv`. Afișați valoarea primită împreună cu tag-ul acesteia folosind `MPI_Status` (status din `MPI_Recv`).

`MPI_ANY_TAG` este folosit în loc de tag-ul mesajului la `MPI_Recv`, indicând faptul că poate primi orice mesaj de la procesul sursă, neținând cont de tag-ul pe care procesul sursă îl pune mesajului la `MPI_Send`.

- **Exercițiul 6 - Bonus (2p):** Se pornește un număr de procese N divizibil cu 4. Se împart procesele în grupe de câte 4 și își trimit în cerc (inel) rangul. Se afișează vechiul rang împreună cu dimensiunea vechiului grup. Se afișează noul rang împreună cu dimensiunea noului grup. Fiecare proces are voie sa comunice doar în grupul lui. Se va folosi `MPI_Comm_split`. Fiecare proces va afișa rangul său din noul grup, numărul grupului și ce număr a primit.

Resurse

- Schelet laborator
- MPI Cheatsheet

Laboratorul 9 - Operații și prelucrări distribuite în MPI

Reduce

Operația de reduce (cunoscută în limbajele funcționale - Haskell, Racket - drept fold) reprezintă un tip de operație prin care elementele unei colecții sunt restrânse / acumulate într-un singur rezultat, printr-o singură operație aplicată între elementele unei colecții (+, *, min, max etc.)

Exemplu:

```
l = [1, 2, 3, 4, 5, 6]
op = +
rezultat = 1 + 2 + 3 + 4 + 5 + 6 = 21
```

În MPI, operația reduce este implementată în funcția MPI_Reduce, însă în cadrul laboratorului vom discuta cum poate fi implementată, de mână, această operație în MPI.

Aici aveți atașate slide-uri, care descriu în detaliu pașii de implementare ai operației reduce: slides

În concluzie, primul proces va avea valoarea finală.

Pseudocod:

```
for (pas = 2; pas <= nr_procese; pas *= 2)
        if (rank % pas == 0)
                primește la procesul cu rank-ul [rank + (pas / 2)]
                adună
        else if (rank % (pas / 2) == 0)
                trimite la procesul cu rank-ul [rank - (pas / 2)]
```

Scan

Operația de scan este similară operației de reduce (acumularea elementelor unei colecții într-un singur rezultat). Diferența față de reduce este că ultimul proces din comunicator deține rezultatul final, practic scan reprezintă inversul lui reduce. În plus, fiecare proces are un rezultat parțial acumulat, în sensul că procesul 0 are valorea P0, procesul 1 are valoarea P0 + P1, procesul 2 are valoarea P0 + P1 + P2, etc.

Exemplu:

```
l = [1, 2, 3, 4, 5, 6]
op = +
rezultat = 1 + 2 + 3 + 4 + 5 + 6 = 21

Pași:
[1, 2, 3, 4, 5, 6]
[1, 3, 3, 4, 5, 6]
[1, 3, 6, 4, 5, 6]
[1, 3, 6, 10, 5, 6]
[1, 3, 6, 10, 15, 6]
[1, 3, 6, 10, 15, 21] -> rezultatul este 21
```

Aici aveți atașate slide-uri, care descriu în detaliu pașii de implementare ai operației scan: slides

Pseudocod:

```
for (pas = 1; pas < nr_procese; pas *= 2)
        if (rank + pas < nr_procese)
                trimite la procesul cu rank-ul [rank + pas]
        if (rank - pas >= 0)
                primește de la procesul cu rank-ul [rank - pas]
                adună
```

Broadcast

Operația de broadcast este o operație prin care un proces trimite o valoare tuturor proceselor din cadrul comunicatorului. Această operație este reprezentată, în MPI, de MPI_Bcast (remember laboratorul 8).

Aici aveți atașate slide-uri, care descriu în detaliu pașii de implementare ai operației broadcast: slides

Pseudocod:

```
for (pas = 1; pas < nr_procese; pas *= 2)
        if (rank < pas and rank + pas < nr_procese)
                trimite la procesul cu rank-ul [rank + pas]
        else if (rank >= pas and rank < pas * 2)
                primește de la procesul cu rank-ul [rank - pas]
```

Pipeline

În domeniul de computer science, un pipeline reprezintă un concept folosit în proiectarea procesoarelor și al echipamentelor hardware prin care se crește numărul de instrucțiuni ce sunt executate într-o unitate de timp.

Acest concept poate fi extins în MPI, unde un proces produce rezultate intermediare, care sunt transmise către procesul următor, care procesează și prelucrează rezultatele primite, pe care le trimite mai departe, către procesul următor, totul repetându-se până la ultimul proces.

Slides cu explicații și exemple: slides

Calculul unui polinom folosind pipeline

Un polinom, în matematică, reprezintă o ecuație de tipul: f(x) = x^2 + 2 * x + 1, unde x reprezintă variabila din ecuație.

În MPI, putem calcula un polinom folosind tehnica pipeline în felul următor: numărul de coeficienți va fi egal cu numărul de procese (care este egal cu numărul de pași din procesul de pipeline). În cazul exemplului de mai sus, o să avem 3 procese, fiecare proces corespunzând cu câte un coeficient din ecuație.

Aici aveți atașate slide-uri, care ilustrează pas cu pas: slides

Sortarea folosind pipeline

Dându-se un vector cu n elemente în ordine aleatoare, dorim să-l sortăm prin metoda pipeline, folosind MPI. Pentru aceasta, vom folosi o rețea cu n + 1 noduri (1 master și n workeri).

Punctul final este momentul în care fiecare proces worker deține stocat unul din numerele din vectorul inițial, iar `rank[i]→saved_value` < `rank[i + 1]→saved_value` pentru orice i din [1, n).

Fiecare worker este inițializat la început cu o valoare (-1 sau depinde de restricții), va primi (n - rank) numere de la rank-ul anterior și va trimite mai departe rank-ului următor (n - rank - 1) numere.

Atunci când primește un număr, worker-ul îl va compara cu valoarea stocată:

- dacă nr < saved_value, atunci salvăm noua valoare și o trimitem pe cea veche procesului următor
- dacă nr >= saved_value, atunci o trimitem direct procesului următor

La final, fiecare worker va trimite către master valoarea lui, iar master-ul va construi vectorul rezultat. Urmăriți slide-urile pas cu pas: slides

Rank Sort

Rank Sort este un algoritm de sortare a unui vector folosindu-se de rangul fiecărui element din vector. Rangul unui element presupune câte numere sunt mai mici decât el în vector.

Pași:

1. Se calculează rangul fiecărui element: pentru fiecare element din vector v[i], unde 0 ⇐ i < N (N - dimensiunea vectorului), se numără câte elemente din vector v[j], j != i sunt mai mici: v[j] < v[i], 0 ⇐ i, j < N, i != j.
2. Elementul de pe poziția "i" va fi pe poziția "rang" în noul vector sortat: q[rank] = v[i].

Algoritmul "Parallel rank sort" are ca scop paralelizarea pasului 1. Rangul fiecărui element poate fi căutat în paralel, adică fiecare proces sau fir de execuție va realiza căutarea rangurilor pentru elementele din intervalul său: [start, end).

Complexitatea temporală a algoritmului este O(n^2), ceea ce face complexitatea în cazul paralelizării pasului 1 a lui să devină O(n^2 / p), p - numărul de procesoare / fire de execuție.

Aici aveți atașate slide-uri, care ilustrează pas cu pas: slides

Exerciții

Schelet laborator

1. **(2p)** Implementați operația de reduction, unde fiecare proces are o valoare proprie, prin care avem o colecție de valori (a nu se folosi MPI_Reduction)
2. **(2p)** Implementați operația de scan, unde fiecare proces are o valoare proprie, prin care avem o colecție de valori (a nu se folosi MPI_Scan)
3. **(2p)** Implementați operația de broadcast, prin care procesul 0 trimite o valoare tuturor proceselor (a nu se folosi MPI_Bcast)
4. **(2p)** Implementați operația de calcul polinomial folosind pipeline, pentru f(5), unde f(x) reprezintă ecuația polinomială (construită prin citirea din fișier existentă în scheletul de cod, unde aveți hint-uri legate de implementare). Pentru testare, există două fișiere de test pentru input (a1.txt și a2.txt) și puteți să vă folosiți de oricare dintre ele, cu precizia că numărul de procese trebuie să fie același cu numărul de coeficienți. Pentru ușurință în testare, puteți folosi regulile run1 și run2 din Makefile.

5. **(2p)** Implementați sortarea prin pipeline, folosind scheletul de laborator și explicațiile din laborator și slide-uri
6. **(2p)** Să se sorteze un vector folosindu-se algoritmul Rank Sort. Procesul master va genera vectorul și îl va popula cu elemente. Va trimite tot vectorul celorlalte procese. Fiecare proces este responsabil pentru calcularea pozițiilor unei anumite părți din vector. La final toate procesele trimit procesului master noile poziții calculate. Procesul master va aranja elementele conform noilor poziții primite.

Resurse suplimentare

Aici aveți atașate resurse suplimentare (slide-uri) legate de conceptele din cadrul acestui laborator, ce pot fi utile pentru testele practice.

Laboratorul 10 - Stabilirea Topologiei. Alegerea Liderului.

In acest laborator vom explora utilizarea MPI in cadrul unei topologii de noduri. Vom invata sa stabilim o topologie, sa alegem un lider (folosind heart-beat) si determinăm numărul de noduri din topologie folosind un algoritm epidemic.

Responsabili: Alexandru Hogea, Cristian Condruz. Multumiri lui Dorinel Filip si Florin Mihalache pentru ajutor.

Organizarea unui cluster

In cazul unui sistem distribuit, elementul cheie este **comunicarea** dintre procese. Aceasta dicteaza performanta sistemului, toleranta la defecte si logica fluxului de transfer de informatii.

Din punct de vedere al arhitecturii, nodurile pot fi organizate in mai multe moduri intr-un cluster:

- **grid** → nodurile sunt organizate pe nivele. Fiecare nod poate comunica cu vecinii pe orizontala si verticala. Fluxul informatiei este vertical si orizontal. Maxim 4 vecini.
- **inel** → nodurile sunt organizate in cerc. Fiecare nod vorbeste cu vecinii din stanga si dreapta. Fluxul informatiei este orizontal. Maxim 2 vecini.
- **arbore** → nodurile sunt organizate ierarhic, sub forma de arbore. Fluxul informatiei este vertical. Numar nelimitat de vecini.
- **graf** → nodurile sunt organizate sub forma unui graf. Fiecare nod poate comunica cu unul sau mai multi vecini. Fluxul informatiei este abstract. Numar nelimitat de vecini.

In cadrul acestui laborator vom explora organizarea unui sistem sub forma de graf generic.

Stabilirea unei topologii

Exista trei moduri prin care o topologie poate fi stabilita in cadrul unui cluster. Nodurile pot cunoaste starea intregului cluster de la bun inceput, pot cunoaste doar vecinii sau nu pot sti nimic despre starea sistemului.

Pentru a stabili topologia, clusterul poate opera in doua moduri:

- nodurile pot transmite starea lor celorlalte moduri de mai multe ori pana cand toate nodurile au topologia stabilita. Numarul de dati necesar asigurarii topologiei intre noduri se numeste **convergenta**. Acest mod este simplu de realizat, dar ineficient
- nodurile aleg un **lider** care sa calculeze topologia sistemului

Problema care se ridica in cazul utilizarii unui lider care sa creeze matricea de topologie, este ca nodurile *nu stiu* calea catre lider, ci doar cine este acest lider. **Arborele de acoperire** rezolva aceasta problema. Pe baza arborelui de acoperire, nodurile trimit informatia pe care o detin catre lider, trecand prin nodurile intermediare care sunt reprezentate de parintii lor in arbore. Nodurile intermediare adauga informatia primita la informatia proprie si o trimit mai departe. In final, liderul are toata informatia sistemului si poate crea matricea de topologie. Matricea este trimisa, apoi, fiecarui nod in parte

In cadrul acestui laborator vom explora stabilirea topologiei prin alegerea liderului.

In cazul retelelor de calculatoare, ruterele si switchurile sunt organizate sub forma unui cluster unde nodurile nu stiu nimic despre starea sistemului la inceput

In cazul alegerii liderului, nu este nevoie sa se cunoasca topologia la inceput, ci doar vecinii cu care nodurile pot interactiona

Alegerea liderului

Clusterele de noduri pot sa fie ierarhizate sau nu. In cadrul clusterelor bazate pe noduri autonome, nu exista un lider bine stabilit, ci contracte clare de comunicare intre noduri. Pe de alta parte, in clusterele care au lider, unul sau mai multe noduri indeplinesc functii elevate. Printre functiile elevate se numara si *stabilirea si distribuirea topologiei* catre celelalte noduri.

Liderul reprezinta un nod sau un grup de noduri ales pe baza unei euristici. Euristica poate tine cont de resursele nodului respectiv, de numarul sau natura proceselor care ruleaza pe el sau pur si simplu de lucruri simple, precum *eticheta sa*.

In cadrul acestui laborator vom explora alegerea liderului bazat pe rank-ul cel mai mare al unui nod. Alegerea liderului se va face pe baza unui algoritm de tipul **heartbeat**.

Alegerea liderului. Heartbeat

Functionarea unui algoritm **heartbeat** se aseamana cu bataile inimii:

1. **sistola** → partea de “expandare”. Informatia este transmisa catre nodurile vecine.
2. **diastola** → partea de “contractie”. Informatia este receptata inapoi de la nodurile vecine.

In cadrul alegerii liderului, algoritmul heartbeat va fi aplicat in felul urmator:

1. nodurile vor trimite rankul lor catre vecini
2. fiecare nod va calcula *liderul local* in functie de maximul dintre rankul propriu si informaia primita de la vecin
3. vecinii vor trimite inapoi liderul lor local catre nodurile de la care au primit mesaj
4. nodurile care au trimis mesajul initial isi vor actualiza valoarea liderului local

Algoritmul se repeta de un numar suficient de ori ca sa se asigure convergenta informatiei.

Un exemplu de rulare a acestui algoritm poate fi observat aici heartbeat_choosing_leader.pdf

Construirea arborelui de acoperire

Dupa ce liderul a fost ales, acesta va **initia construirea arborelui de acoperire**.

Scopul arborelui de acoperire este ca fiecare nod sa stie, in orice moment de timp, cum sa ajunga catre lider.

Arborele de acoperire se reprezinta, cel mai usor, printr-un *vector de tati*. Constuirea acestui arbore se realizeaza printr-un algoritm de tip **unda - ecou**.

Construirea arborelui de acoperire. Unda - ecou

Algoritmul unda are doua parti:

1. **Unda** → toate nodurile, cu exceptia liderului asteapta sa primeasca un mesaj. Liderul initiaza unda. La primirea **primei sonde**, fiecare nod isi noteaza parintele si trimite un mesaj tuturor vecinilor, mai putin celui de la care a primit (părintele său)

2. **Ecou** → se asteapta primirea unui mesaj de la toti vecinii, *mai puțin de la părinte*. Informatiile primite de la vecini sunt agregate intr-un vector de tati si acesta este trimis către părinte.

La final, liderul va detine vectorul de tati final, care reprezinta **arborele de acoperire**. Arborele de acoperire este apoi distribuit intregului sistem

Verificarea topologiei

Dupa ce liderul a fost ales si topologia a fost distribuita catre toate nodurile, este util ca informatia sa fie validata. Validarea poate implica procese foarte complexe, insa pentru acest laborator vom folosi o validare bazata pe numarul de noduri prezente in cluster.

Calcularea numarului de noduri se poate realiza folosind un algoritm de tipul **epidemic**.

Calcularea numarului de noduri. Epidemic

Functionalitatea unui algoritm de tipul **epidemic** se aseamana cu raspandirea unei boli in care exista un pacient 0:

1. “pacientul 0” transmite informatia la vecinii sai
2. vecinii pacientului 0 devin “infectati” si transmit si ei, mai departe, informatia, catre vecinii lor

Pentru calcularea numarului de noduri, pacientul 0 va fi nodul lider. Acesta va detine o valoare de inceput, **1**. Toate celelalte noduri detin valoarea **0**. Nodul lider va realiza media aritmetica dintre valoarea sa si valoarea vecinilor sai. Informatia va fi sincronizata in ambele parti. Celalte noduri vor executa aceeasi operatie cu vecinii lor.

Algoritmul se repeta pana cand se atinge convergenta solutiei. La final, toate nodurile vor detine o aceeasi valoare subunitara. Impartirea lui 1 la aceasta valoare va rezulta in numarul de noduri al clusterului.

Un exemplu de rulare a acestui algoritm poate fi observat aici epidemic_counting_nodes.pdf

Exercitii

Pornind de la scheletul de laborator, va trebui sa realizati urmatoarele sarcini:

1. Aflati nodul lider al clusterului folosind un algoritm *heartbeat* (**3p**)
2. Realizati arborele de acoperire, plecand din lider, folosind un algoritm *unda-ecou* (**3p**)
3. Asigurati-va ca numarul de elemente din arborele de acoperire a fost stabilit bine prin calcularea numarului de noduri folosind un algoritm *epidemic* (**3p**)
4. Folosind arborele de acoperire, trimiteti catre lider configuratia fiecarui nod si realizati in lider matricea de topologie a clusterului. Distribuiti topologia catre toate nodurile (**1p**)

MPI Cheat Sheet

Arguments from the main function
Called at the start of any MPI program

int MPI_Init(↕ int *, ↕ char *)**

&argc NULL
&argv NULL

Called at the end of any MPI program

int MPI_Finalize()

Gives the number of tasks

int MPI_Comm_size (↓ MPI_Comm, ↑ int *)

MPI_COMM_WORLD

&num_tasks

Gives the id (rank) of the current (calling) task

int MPI_Comm_rank (↓ MPI_Comm, ↑ int *)

MPI_COMM_WORLD

&rank

Synchronizez all tasks at the call of the barrier

int MPI_Barrier (↓ MPI_Comm comm)

MPI_COMM_WORLD

Send from buffer **b**, **c** elements of data type d to rank r. The communication is marked with tag **t**.
The function is blocking, b can safely be used after it but data may not have yet been delivered.

int MPI_Send(↓ void *b, ↓ int c, ↓ MPI_Datatype d, ↓ int reiceiver, ↓ int t, ↓ MPI_Comm)

v	num_el(v)	MPI_INT	[0, **num_tasks**)	[0, ..)	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR			
&a		MPI_FLOAT			
v+5		MPI_LONG			

Receive in buffer **b**, **c** elements of data type d from rank r. The communication is marked with tag **t**.
The function is bloking, b can be safely used and the data was delivered.

int MPI_Recv(↑ void *b, ↓ int c, ↓ MPI_Datatype d, ↓ int sender, ↓ int t, ↓ MPI_Comm, ↑ MPI_Status *)

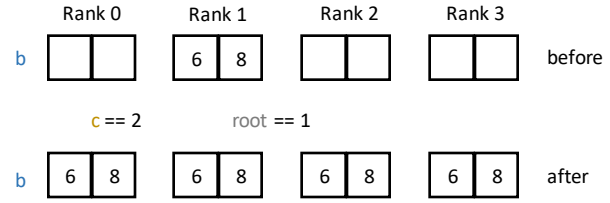
v	num_el(v)	MPI_INT	[0, **num_tasks**)	MPI_COMM_WORLD	
&v[3]	[0,..)	MPI_CHAR	MPI_ANY_SOURCE		
&a		MPI_FLOAT		&Stat	
v+5		MPI_LONG	[0, ..)	MPI_STATUS_IGNORE	
			MPI_ANY_TAG	***Stat.MPI_SOURCE, Stat.MPI_TAG***	

Made by Cristian Chilipirea

Sends (Broadcasts) **c** elements of data type d from buffer **b** from rank r to all other tasks in buffer **b**.
All tasks have to call this function with the same value for root.

int MPI_Bcast (↕ void *b, ↓ int c, ↓ MPI_Datatype d, ↓ int root, ↓ MPI_Comm)

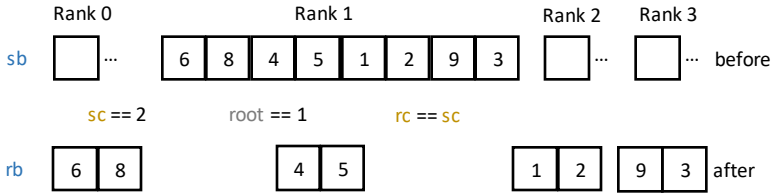
v	num_el(v)	MPI_INT	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR	[0, **num_tasks**)
&a		MPI_FLOAT	
v+5		MPI_LONG	



Splits the elements from **sb** of datatype sd on rank root in num_tasks chunks of size **sc**.
Every task receives its appropriate chunk in **rb**. For simplicity **sc** == **rc**, sd == rd.
All tasks have to call this function with the same value for root.

int MPI_Scatter (↓ void *sb, ↓ int sc, ↓ MPI_Datatype sd, ↑ void *rb, ↓ int rc, ↓ MPI_Datatype rd, ↓ int root, ↓ MPI_Comm)

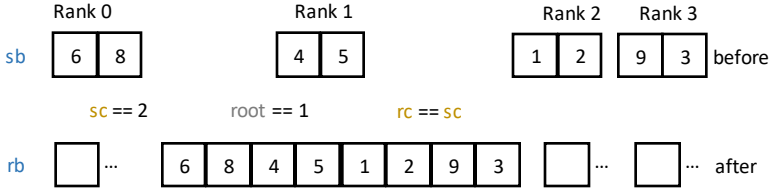
v	num_el(v)/**num_tasks**	MPI_INT	v	num_el(v)/**num_tasks**	MPI_INT	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR	&v[3]	[0,..)	MPI_CHAR	[0, **num_tasks**)
&a		MPI_FLOAT	&a		MPI_FLOAT	
v+5		MPI_LONG	v+5		MPI_LONG	



Gathers **sc** elements from all **sb** of datatype sd on all tasks and places the **num_tasks** chunks of size **rc** in **rb** on task of rank root.
Every task sends its appropriate chunk in **rb**. For simplicity **sc** == **rc**, sd == rd.
All tasks have to call this function with the same value for root.

int MPI_Gather (↓ void *sb, ↓ int sc, ↓ MPI_Datatype sd, ↑ void *rb, ↓ int rc, ↓ MPI_Datatype rd, ↓ int root, ↓ MPI_Comm)

v	num_el(v)/**num_tasks**	MPI_INT	v	num_el(v)/**num_tasks**	MPI_INT	MPI_COMM_WORLD
&v[3]	[0,..)	MPI_CHAR	&v[3]	[0,..)	MPI_CHAR	[0, **num_tasks**)
&a		MPI_FLOAT	&a		MPI_FLOAT	
v+5		MPI_LONG	v+5		MPI_LONG	



PThread Cheat Sheet

Made by Cristian Chilipirea

```
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

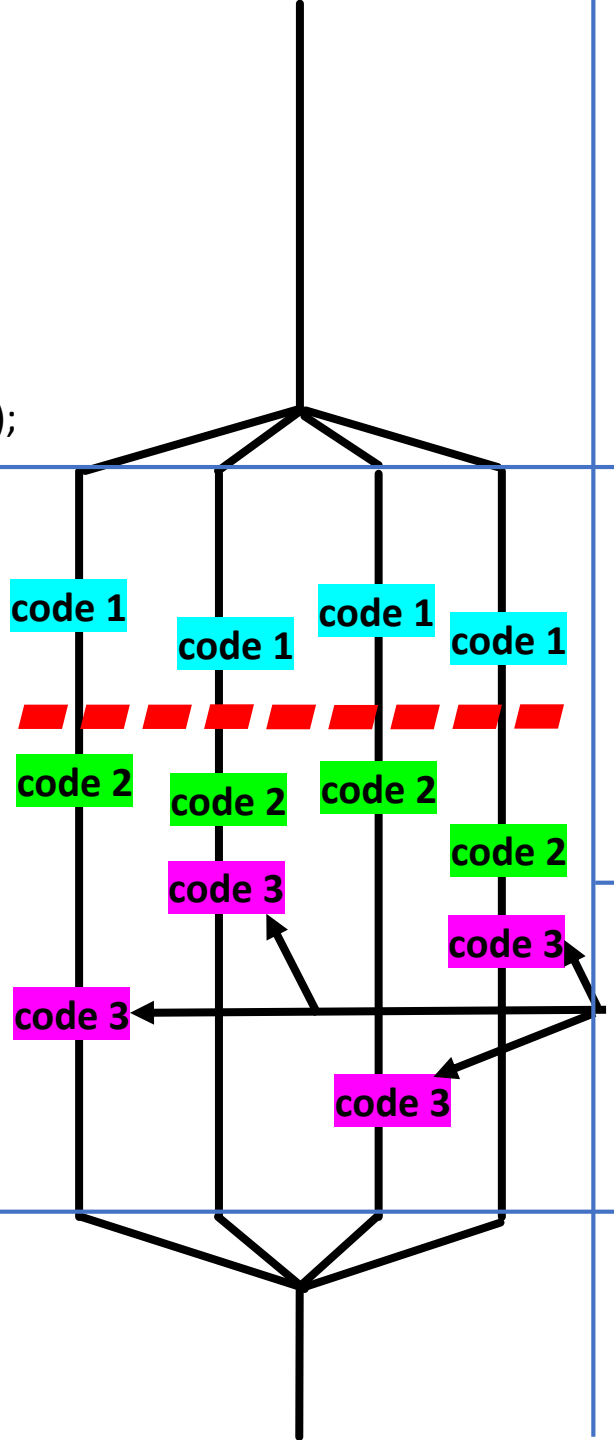
pthread_barrier_t barrier;
pthread_barrier_init(&barrier, NULL, num_threads);

pthread_t thread;
pthread_create(&thread, NULL, threadFunction, arg);
```

```
#include <pthread.h>
```

```
void * threadFunction(void* arg)
{
    code 1
    pthread_barrier_wait(&barrier);
    code 2

    pthread_mutex_lock(&mutex);
    code 3
    pthread_mutex_unlock(&mutex);
}
```



Any **code 2** must execute **after all code 1**

When **code 3** starts execution it must end before it can start on a different thread

```
pthread_join(thread, NULL);
pthread_barrier_destroy(&barrier);
pthread_mutex_destroy(&mutex);
```

Compiling:
gcc main.c –fpthread
Number of cores:
cat /proc/cpuinfo